

Justificaciones de diseño TO-BE

Aplicación de principios SOLID

Sistema Hacienda — Fase 3: diseño corregido acumulativo

Metodología de rediseño

La metodología de rediseño consistió en analizar los hallazgos de manera individual, proponer una solución estructural para cada problema y, una vez reunidas las decisiones, revisar la arquitectura como un conjunto para comprobar su coherencia y la aplicación acumulativa de los principios SOLID.

Este documento se concentra en justificar las decisiones de diseño. Para ampliar el problema señalado en cada hallazgo, consultar la tabla de hallazgos del inventario, donde se encuentra la descripción completa, la ubicación y la línea de código asociada.

Criterio de presentación: las referencias a clases, interfaces, métodos y relaciones se explican principalmente en lenguaje de diseño y de negocio. Se conservan los nombres técnicos necesarios para la trazabilidad con el código, pero se evita reproducir bloques extensos de implementación cuando la misma decisión puede justificarse de forma narrativa.

H-01 — Hacienda concentra responsabilidades de áreas distintas

Problemas señalados en el hallazgo

Hacienda concentra en una sola clase la administración de potreros, incorporación y alimentación de reses, ventas y gestión de vacunas, por lo que cambios de áreas distintas afectan el mismo módulo.

Solución aprobada

Cambio detallado

- PotreroService: crear/buscar potreros e incorporar reses.
- ResService: alimentar reses y mantener los eventos de peso que acompañan ese caso de uso.
- VentaService: ejecutar la venta de reses.
- VacunaService: crear y aplicar vacunas y mantener los publishers asociados a vacunación.

Las operaciones no se “reescriben” en H-01: se trasladan íntegramente. Junto con cada operación viajan sus reglas, validaciones, cálculos, excepciones, mensajes, eventos, orden de ejecución y efectos sobre el estado. De esta manera H-01 corrige SRP sin alterar la conducta observable.

- Reglas: si la operación trasladada usaba ReglaRes, ReglaVacuna o ReglaPotrero, esa dependencia sigue al nuevo responsable. H-01 no cambia los valores ni la semántica de esas reglas.
- Publishers: PublisherPesoMin/PublisherPesoVenta acompañan a ResService; PublisherVacunaVencida/PublisherVacunacionCompletada acompañan a VacunaService.
- Contratos: IVacunacion e ICreacionVacuna pasan a VacunaService; IVentaRes pasa a VentaService porque Hacienda deja de ejecutar esos casos de uso.
- Estado de Hacienda: las colecciones y accesorios existentes no se cambian por H-01. No se modifica su encapsulamiento.

Por qué esa frontera

La separación se realiza por razón de cambio del negocio, no por tamaño de clase. Potreros, manejo animal, ventas y vacunación responden a políticas distintas y pueden evolucionar de forma independiente. Además, el proyecto ya poseía los cuatro servicios, por lo que reutilizar esas fronteras evita crear un sistema paralelo contrario a la petición del cliente.

Qué se gana

- menor radio de impacto de cada cambio
- pruebas más focalizadas
- mayor cohesión funcional
- menor riesgo de regresión entre áreas no relacionadas
- base coherente para resolver posteriormente los hallazgos internos de los Services sin mantener Hacienda como clase central de lógica

Impacto para negocio y mantenimiento

La ganancia no es solamente una clase más pequeña. Antes, un cambio sanitario o comercial podía obligar a revalidar una clase que también contenía potreros, alimentación y ventas. Después de H-01, el radio de cambio se concentra en el servicio de la política afectada. Esto reduce horas de prueba de regresión, costo de mantenimiento y riesgo de introducir fallas en procesos no relacionados.

Las colecciones y asociaciones de estado conservadas en Hacienda permanecen negras.

H-02 — Vacunación exige modificar lógica existente ante nuevas categorías o cambios de reglas

Problemas señalados en el hallazgo

La lógica de vacunación distingue explícitamente Ternero, Cebon y Novillo para aplicar límites sanitarios y mantiene reglas de cantidad de lote dentro del flujo principal, obligando a modificar código existente cuando cambian categorías o límites.

En el sistema original, la lógica de vacunación distinguía de manera explícita si la res era Ternero, Novillo o Cebon para decidir qué límites aplicar. En el mismo flujo también se comprobaba directamente que la cantidad del lote fuera mayor que cero y no superara cien unidades. Esta forma de decisión hacía que el método tuviera que modificarse cada vez que aparecía una nueva categoría o cambiaban los límites sanitarios.

Solución aprobada

Cambio detallado

Decisión 1 — una capacidad pequeña para eliminar la enumeración de tipos

Se crea `IReglasVacunacionRes`. Ternero, Cebon y Novillo —clases ya existentes— implementan esta capacidad. `VacunaService` deja de conocer las tres clases concretas para obtener/ejecutar los límites. No se agrega comportamiento abstracto a `Res` porque la vacunación es una capacidad específica y la jerarquía `Res` será revisada posteriormente por los hallazgos LSP.

`IReglasVacunacionRes` se define como un contrato pequeño que reúne únicamente las capacidades que el proceso de vacunación necesita de una res: comprobar los límites de vacunas bacterianas, comprobar los límites de vacunas vivas y determinar si el esquema de vacunación está completo. De esta manera,

VacunaService trabaja con una capacidad común y deja de depender de las clases concretas Ternero, Cebon y Novillo.

Decisión 2 — separar las reglas por categoría para permitir crecimiento ordenado

Los seis máximos que estaban centralizados en ReglaVacuna se redistribuyen en clases de regla específicas, dentro de Reglas/ReglasVacunacion. Esta separación no constituye un patrón de diseño: no existe un selector, una colección de estrategias ni sustitución dinámica. Son clases de reglas de negocio con una responsabilidad claramente delimitada.

- ReglaVacunacionTernero conserva los límites de negocio de tres vacunas bacterianas y una vacuna viva, y concentra la validación de esos límites y la determinación de cuándo el esquema del ternero está completo.
- ReglaVacunacionCebon conserva los límites de una vacuna bacteriana y cuatro vacunas vivas, y concentra la validación de esos límites y la determinación de cuándo el esquema del cebón está completo.
- ReglaVacunacionNovillo conserva los límites de dos vacunas bacterianas y dos vacunas vivas, y concentra la validación de esos límites y la determinación de cuándo el esquema del novillo está completo.

Las clases Ternero/Cebon/Novillo implementan IReglasVacunacionRes y delegan cada operación en su clase de reglas correspondiente. Así, VacunaService solo usa la interfaz y no necesita un selector de “políticas”.

Ternero, Cebon y Novillo ofrecen la capacidad definida por IReglasVacunacionRes. Cada una de estas clases delega las decisiones sanitarias en su regla específica —ReglaVacunacionTernero, ReglaVacunacionCebon o ReglaVacunacionNovillo—. VacunaService utiliza únicamente el contrato común, por lo que no necesita conocer cuál de las tres implementaciones concretas está atendiendo la solicitud.

Decisión 3 — separar la regla de lote

La cantidad del lote responde a una razón de cambio diferente de los máximos por categoría de res, por lo que se crea ReglaLoteVacunas en el mismo bloque de reglas. No se crea una interfaz porque actualmente existe una única regla de lote y no hay una dependencia que necesite sustitución.

ReglaLoteVacunas concentra exclusivamente la política de cantidad por lote. La regla mantiene el mínimo observable de una unidad y el máximo de cien; cuando la cantidad no cumple esos límites se conservan las mismas excepciones y mensajes que tenía el sistema original. El cambio consiste en localizar esa política en una clase con una única razón de cambio, no en alterar la regla de negocio.

La literalidad de los mensajes se conserva. El valor 100 sigue siendo 100 y el mínimo observable sigue siendo mayor que 0; solamente cambia el lugar en el que la regla se almacena y ejecuta.

Decisión 4 — tratamiento de una Res sin reglas de vacunación

VacunaService realiza una comprobación explícita antes de evaluar límites. Si la Res recuperada no implementa IReglasVacunacionRes, se lanza una InvalidOperationException clara indicando que no existen reglas de vacunación definidas para esa res. Esta guardia evita que una categoría futura termine usando implícitamente máximos 0 y produzca una falla difícil de interpretar.

Antes de aplicar las reglas sanitarias, VacunaService comprueba que la res disponga de una implementación de IReglasVacunacionRes. Si una categoría futura no declara esa capacidad, el servicio produce de forma explícita la misma clase de error prevista por el diseño, indicando que no existen reglas de vacunación definidas para esa res.

Esta guardia NO se registra como un hallazgo formal nuevo. En el AS-IS el propio Potrero solo construye Ternero, Cebon y Novillo, y los tres implementan la nueva interfaz; por tanto, la guardia no altera ninguna ruta

observable actual. Es una condición explícita del nuevo contrato de extensibilidad. Tampoco se presenta como corrección de los hallazgos LSP H-09/H-10/H-11, que corresponden a la propiedad Edad y se resolverán por separado.

Reutilización necesaria en PublisherVacunacionCompletada

Al sacar los seis máximos de ReglaVacuna, PublisherVacunacionCompletada no debe duplicarlos ni volver a discriminar Ternero/Cebon/Novillo. Reutiliza IReglasVacunacionRes para consultar EsquemaCompleto. Para las tres categorías actuales conserva exactamente el mismo bool y los mismos mensajes/eventos. Esto anticipa la solución del H-27; cuando se llegue formalmente a ese hallazgo se registrará como reutilización ya implementada.

Qué permanece sin tocar

- ReglaVacuna conserva periodo_min_bac_aplic = 2 y periodo_max_bac_aplic = 4, porque esas reglas no son el eje de variación de H-02.
- Conteos, validación de vacuna duplicada, verificación de vencimiento, actualización del inventario, suscripción/disparo de eventos y mensaje final permanecen con la misma semántica y orden.

Por qué esa frontera

- IReglasVacunacionRes representa únicamente la capacidad necesaria para vacunación y evita ensanchar la clase base Res.
- Las reglas por categoría se separan porque se prevé que crecerán de forma independiente durante los próximos años; agruparlas en ReglaVacuna volvería a concentrar cambios heterogéneos.
- ReglaLoteVacunas se separa porque el tamaño de lote no depende de la categoría de res y tiene una razón de cambio distinta.
- VacunaService no conoce ReglaVacunacionTernero/Cebon/Novillo; solo conoce la interfaz y la regla general de lote, por lo que agregar una nueva categoría no requiere modificar la selección central.
- No se utiliza un patrón: no hay contexto, selector, colección de estrategias ni factoría.

Cómo aplica SOLID

- OCP — principal: VacunaService deja de enumerar Ternero/Cebon/Novillo. Una categoría futura extiende el comportamiento implementando IReglasVacunacionRes y aportando su clase de reglas, sin cambiar la lógica central de selección por tipo.
- SRP — beneficio secundario: cada clase de regla tiene una razón de cambio sanitaria concreta; ReglaLoteVacunas cambia por política de lote y cada ReglaVacunacionX cambia por política de esa categoría. La justificación del hallazgo, sin embargo, sigue siendo OCP.
- LSP: la nueva guardia no resuelve los hallazgos LSP existentes. Evita una falla implícita para tipos que no declaren la capacidad de vacunación, pero H-09/H-10/H-11 continúan abiertos.

Preservación del comportamiento observable

Regla/efecto	AS-IS	TO-BE
Ternero	3 bacterianas / 1 viva	3 bacterianas / 1 viva
Cebon	1 bacteriana / 4 vivas	1 bacteriana / 4 vivas
Novillo	2 bacterianas / 2 vivas	2 bacterianas / 2 vivas
Lote	cantidad > 0 y máximo 100	cantidad > 0 y máximo 100
Mensajes/eventos	Textos actuales	Los mismos textos para las rutas actuales

H-03 — Hacienda construye directamente dependencias concretas

Problemas señalados en el hallazgo

Hacienda construye directamente Potrero, Venta, Bacteriana, Viva y varios publishers, quedando acoplada a detalles de construcción de procesos que pertenecen a responsabilidades diferentes.

Hacienda no solo coordinaba varios casos de uso; también instanciaba directamente Potrero, Venta, Bacteriana y Viva. La revisión de las líneas citadas muestra además que inicializaba directamente PublisherVacunacionCompletada, PublisherVacunaVencida, PublisherPesoMin y PublisherPesoVenta. El efecto era que Hacienda conocía detalles de construcción pertenecientes a procesos distintos, dificultando aislarla y haciendo que cambios en esas implementaciones pudieran propagarse hacia ella.

En el AS-IS, Hacienda era responsable de construir directamente los objetos asociados a potreros, ventas y vacunas, además de los publishers utilizados en los procesos de vacunación y peso. Esta concentración de decisiones de construcción explica por qué la clase quedaba acoplada a detalles que no pertenecían a una única responsabilidad de negocio.

Solución aprobada

Cambio detallado

Solución y relocalización completa

Construcción original	Nuevo responsable	Condición de preservación
Construcción directa de Potrero	PotreroService	continúa formando parte del flujo de creación de potreros
Construcción directa de Venta	VentaService	continúa formando parte del flujo de venta
Construcción directa de Bacteriana y Viva	VacunaService	continúan formando parte de la creación de vacunas; su extensibilidad se revisa en H-22
PublisherPesoMin / PublisherPesoVenta	ResService	continúan ejecutándose en los flujos de peso/alimentación
PublisherVacunaVencida / PublisherVacunacionCompletada	VacunaService	continúan ejecutándose en el flujo sanitario

La regla de implementación es estricta: relocalizar no significa eliminar ni simplificar. Se deben conservar los mismos constructores, parámetros, orden de efectos, eventos, excepciones, mensajes, reglas de negocio y resultados observables del sistema original.

Por qué se elimina la dependencia en lugar de crear una interfaz

DIP no obliga a crear una interfaz para cada uso de new. En esta frontera, Hacienda ya no debe crear esos objetos. Introducir ICreadorPotrero, ICreadorVenta o interfaces equivalentes dentro de Hacienda reintroduciría precisamente las responsabilidades que H-01 retiró. La frontera más estable es que Hacienda no conozca esos detalles de construcción.

- Criterio: si una dependencia concreta no pertenece a la responsabilidad del módulo de alto nivel, primero se elimina; solo se introduce una abstracción cuando la colaboración debe permanecer.
- Restricción de patrones: no se introduce Factory, Strategy ni otro patrón. La corrección es redistribución de responsabilidades y eliminación de dependencias innecesarias.

Por qué esa frontera

- PotreroService: la construcción del potrero cambia junto con el caso de uso de gestión de potreros.
- VentaService: la construcción de Venta cambia junto con el proceso comercial.
- VacunaService: la creación de Bacteriana/Viva y los publishers sanitarios pertenecen al proceso de vacunas.
- ResService: los publishers de peso pertenecen al comportamiento de manejo/alimentación de reses.

Qué se gana

- Hacienda deja de conocer detalles de construcción pertenecientes a procesos que ya no coordina.
- Los cambios de construcción quedan localizados en la frontera funcional correspondiente.
- Se reduce la propagación de cambios hacia Hacienda.
- Se facilita aislar Hacienda en pruebas porque deja de inicializar colaboradores de ventas, vacunas y eventos.
- Se evita crear abstracciones artificiales que aumentarían complejidad sin conservar una colaboración necesaria.

Matriz explícita de inversión/eliminación de dependencias

Campo exigido	H-03
Módulo de alto nivel	Hacienda.
Módulo de bajo nivel / detalle concreto	Potrero, Venta, Bacteriana, Viva y los publishers concretos que Hacienda construía.
Abstracción que desacopla	No aplica en esta frontera: la dependencia no debe existir. Crear una interfaz solo para conservarla reintroduciría una responsabilidad eliminada por H-01.
Composition root	La dependencia Hacienda → detalles se elimina, por lo que no existe una construcción que resolver para esa relación. Las dependencias que permanecen en los servicios resultantes se conectan en p_mvcHacienda/Program.cs, método Main, mediante el contenedor configurado con builder.Services.

Esta excepción se documenta expresamente para no confundir DIP con «crear una interfaz para cada new». Cuando la colaboración no pertenece al módulo de alto nivel, eliminarla es una frontera más estable que invertirla artificialmente.

H-04 — Potrero depende de tipos concretos para crear reses

Problemas señalados en el hallazgo

Potrero conoce directamente los constructores de Ternero, Cebon y Novillo y decide mediante condiciones qué tipo crear, por lo que depende de implementaciones concretas.

Potrero.anadir_res() conoce directamente cómo construir Ternero, Cebon y Novillo. Además, la misma operación determina mediante switch el rango de edad que corresponde a cada tipo. Mover solamente el new a otra clase conservando un switch central no se acepta para el TO-BE, porque mantendría una enumeración rígida de las variantes y trasladaría el problema OCP a otra ubicación.

En el AS-IS, la operación de incorporación de reses en Potrero contenía dos decisiones cerradas. Primero elegía el rango de edad a partir del tipo de potrero y, después, seleccionaba directamente si debía crear un Ternero, un Cebon o un Novillo. Por eso, incorporar una nueva categoría exigía modificar el mismo flujo central en más de un punto.

Solución aprobada

Cambio detallado

Frontera elegida

Se introduce `IReglasTipoPotrero` como abstracción estable de las operaciones que Potrero necesita para incorporar una res: validar que la edad corresponda al tipo de potrero y construir la Res concreta. Potrero deja de conocer Ternero, Cebon y Novillo para estas decisiones.

`IReglasTipoPotrero` expresa las dos capacidades que Potrero necesita para incorporar una res sin conocer clases concretas: validar que la edad sea compatible con el tipo de potrero y crear la Res correspondiente a esa política. El contrato recibe la información necesaria de la res y devuelve una instancia tratada mediante el tipo base Res.

Nuevas implementaciones de reglas del negocio:

- `ReglaTipoPotreroTernero`: conserva el rango 0–12 meses y crea Ternero(nombre, peso, edad).
- `ReglaTipoPotreroCebon`: conserva el rango 13–48 meses y crea Cebon(nombre, peso, edad).
- `ReglaTipoPotreroNovillo`: conserva el rango ≥ 49 meses y crea Novillo(nombre, peso, edad).

Cómo queda Potrero

Potrero conserva una referencia privada y de solo lectura a `IReglasTipoPotrero`, recibida durante su construcción. La identificación del potrero continúa siendo responsabilidad de la entidad, mientras que la validación de edad y la creación del subtipo de res se delegan a la regla recibida. De esta forma, la operación de incorporación ya no contiene conocimiento directo de Ternero, Cebon o Novillo.

El cambio es estructural. Los rangos de edad, constructores concretos, eventos, mensajes, conteos, catch/wrapping de excepciones y secuencia de efectos deben producir exactamente el mismo resultado observable.

Cómo se evita el switch al seleccionar las reglas

La relación entre el tipo textual que ya recibe la aplicación (por ejemplo, "ternero", "cebon" o "novillo") y la implementación de `IReglasTipoPotrero` se configura fuera del dominio, en Program. `PotreroService` consulta esa configuración por tipo de potrero; no contiene switch, enum ni una cadena de if por subtipo.

En Program, que actúa como composition root, se crean las implementaciones de reglas para ternero, cebón y novillo y se organizan por el valor de `TipoPotrero` que cada una declara. `PotreroService` utiliza esa configuración para obtener la abstracción correspondiente al mismo texto que ya recibe desde la aplicación. La selección concreta queda así fuera del dominio y no requiere un switch dentro de Potrero.

Este mapa se considera configuración del composition root, no un patrón de diseño. No existe selector con switch, jerarquía de fábricas, colección de estrategias dentro del dominio ni lógica condicional basada en los concretos.

Por qué esa frontera

- Potrero: es el módulo de mayor nivel que aplica la política de incorporación y conserva el estado del potrero; no debe conocer constructores concretos.

- IReglasTipoPotrero: define únicamente lo que Potrero necesita de un tipo para validar e incorporar una res.
- ReglaTipoPotrero...: concentra los detalles que cambian cuando cambia una categoría: su rango de edad y la clase concreta que se crea.
- Program: es el lugar apropiado para conocer implementaciones y configurarlas; modificar la composición no obliga a modificar Potrero.

Aplicación de DIP

Antes del rediseño, Potrero dependía directamente de Ternero, Cebon y Novillo para realizar la incorporación. En el TO-BE, Potrero depende de IReglasTipoPotrero; las implementaciones concretas de esa interfaz son las que conocen cómo validar y construir la categoría correspondiente. La dependencia de alto nivel queda dirigida hacia la abstracción y los detalles concretos quedan al otro lado de la frontera.

El módulo de mayor nivel deja de depender de los detalles concretos y depende de una abstracción. Las implementaciones específicas conocen los constructores concretos. Esta es la inversión de dependencia que exige H-04.

Matriz explícita de inversión de dependencias

Campo exigido	H-04
Módulo de alto nivel	Potrero: mantiene el estado y aplica el flujo general de incorporación.
Módulo de bajo nivel / detalle concreto	ReglaTipoPotreroTernero, ReglaTipoPotreroCebon y ReglaTipoPotreroNovillo; en ellas quedan los detalles de construcción de Ternero/Cebon/Novillo.
Abstracción que desacopla	IReglasTipoPotrero.
Composition root	p_mvcHacienda/Program.cs, método Main: construye las reglas de edad/peso y ReglaTipoPotreroX, forma el mapa por TipoPotrero y registra ITiposPotreroDisponibles; el contenedor resuelve PotreroService, que suministra la abstracción al construir Potrero.

Por qué no se aceptó la alternativa con ConstruccionesRes + switch

Una clase ConstruccionesRes que recibiera el enum y ejecutara un switch solo movería la enumeración de concretos. Potrero quedaría desacoplado para DIP, pero el nuevo módulo seguiría requiriendo modificación cada vez que apareciera una categoría. Esa solución sería frágil frente a H-05 y no cumple la regla del proyecto de no utilizar switch.

Qué se gana

- Potrero deja de conocer Ternero, Cebon y Novillo como detalles de construcción.
- Los rangos de edad y la construcción quedan concentrados por categoría y pueden crecer de manera independiente.
- Agregar una nueva categoría no obliga a añadir una rama dentro de Potrero.
- La abstracción permite pruebas aisladas de Potrero con una implementación controlada de IReglasTipoPotrero.

- La misma frontera se reutiliza posteriormente para la carga de reses en persistencia (H-14), evitando duplicar el mismo eje de variación.

H-05 — Potrero está cerrado alrededor de tres categorías concretas

Problemas señalados en el hallazgo

Potrero mantiene un catálogo cerrado de Ternero, Cebon y Novillo mediante decisiones condicionales y un enum interno; agregar una categoría obliga a modificar la clase.

Potrero estaba construido alrededor de Ternero, Cebon y Novillo mediante decisiones condicionales y un enum anidado `l_tipos_potreros`. H-04 ya retiró los switch y la construcción concreta de Potrero. Sin embargo, mantener el enum dentro de Potrero conservaría un catálogo cerrado: agregar una nueva categoría todavía exigiría modificar la propia clase.

En el AS-IS, Potrero mantenía internamente un catálogo cerrado con las categorías ternero, novillo y cebón. Ese catálogo se utilizaba tanto para determinar rangos de edad como para decidir qué clase concreta construir. Por ello, agregar una nueva categoría requería editar la propia clase Potrero y ampliar sus decisiones condicionales.

Solución aprobada

Cambio detallado

Solución acumulativa con H-04

No se crea una segunda arquitectura para H-05. Se reutiliza `IReglasTipoPotrero` y las tres implementaciones creadas en H-04. H-05 completa la apertura a extensión eliminando el enum central y trasladando la identidad del tipo a cada implementación.

`IReglasTipoPotrero` se amplía para que cada implementación informe también su propio `TipoPotrero`, además de validar la edad y crear la Res correspondiente. Con esta decisión, la identidad de cada categoría deja de pertenecer a un catálogo central dentro de Potrero y pasa a formar parte de la implementación que conoce esa categoría.

- `ReglaTipoPotreroTernero`: implementa `TipoPotrero` = "ternero" y conserva exactamente las reglas/constructor aprobados en H-04.
- `ReglaTipoPotreroCebon`: implementa `TipoPotrero` = "cebon" y conserva exactamente las reglas/constructor aprobados en H-04.
- `ReglaTipoPotreroNovillo`: implementa `TipoPotrero` = "novillo" y conserva exactamente las reglas/constructor aprobados en H-04.

Cómo queda Potrero

Potrero ya no posee `l_tipos_potreros`. Conserva la referencia a `IReglasTipoPotrero` y el valor público `Tipo_potrero` pasa a ser el `TipoPotrero` informado por esa regla. Los textos visibles actuales se mantienen exactamente como ternero, cebon y novillo.

Potrero mantiene como única fuente de verdad la regla de tipo recibida durante su construcción. La propiedad pública `Tipo_potrero` obtiene su valor desde `IReglasTipoPotrero`, por lo que el texto que identifica la categoría y las reglas efectivamente aplicadas no pueden divergir. La entidad conserva la identificación y la referencia a la abstracción, pero deja de almacenar un segundo valor de tipo independiente.

Además, se elimina el atributo privado `tipo_potrero`. `Tipo_potrero` queda como propiedad derivada de `reglas_tipo_potrero.TipoPotrero`, de modo que existe una sola fuente de verdad y no pueden divergir el texto del tipo y las reglas efectivamente asociadas al Potrero.

Entrada desde controlador y servicio

La interfaz web puede seguir enviando exactamente los mismos valores textuales que antes producía el enum. `PotreroController` y `PotreroService` siguen recibiendo `tipo` como `string`. Desde H-16, `PotreroService` no conserva el mapa: solicita la regla correspondiente a `ITiposPotreroDisponibles`, cuya implementación `TiposPotreroDisponibles` mantiene la configuración por `TipoPotrero`, sin switch.

La capa web continúa enviando los mismos valores textuales —ternero, cebon y novillo— y `PotreroService` mantiene la misma entrada pública. La diferencia es interna: el servicio solicita a `ITiposPotreroDisponibles` la regla asociada al texto recibido y construye el Potrero con esa abstracción. El resto del flujo observable permanece igual.

Composition root sin catálogo de dominio dentro de Potrero

Program conoce las implementaciones concretas y configura la relación entre sus valores `TipoPotrero` y las abstracciones. Desde H-16, ese mapa se entrega a `TiposPotreroDisponibles`; `PotreroService` solo consume `ITiposPotreroDisponibles`. Modificar la composición al incorporar una extensión es esperado; el algoritmo estable de Potrero no cambia.

Program configura las tres reglas actuales de tipo de potrero y las relaciona con el valor textual que cada una declara. Esa configuración se entrega posteriormente a `TiposPotreroDisponibles`, de modo que ampliar el sistema con otra categoría consiste en registrar una nueva implementación en el composition root sin modificar la lógica estable de Potrero.

Por qué no se usa enum

Un enum representa un conjunto cerrado de alternativas. Si el objetivo del hallazgo es que nuevas categorías puedan incorporarse sin modificar Potrero, mantener `I_tipos_potreros` dentro de la clase conservaría una frontera cerrada. Por eso el TO-BE evita enums para esta variación del dominio y hace que cada extensión declare su propia identidad mediante `TipoPotrero`.

Preservación de comportamiento observable

- Los valores enviados por el formulario siguen siendo exactamente "ternero", "cebon" y "novillo".
- Las vistas que muestran `@potrero.Tipo_potrero` siguen mostrando exactamente esos mismos textos.
- Persistencia sigue escribiendo exactamente los mismos valores de tipo después del identificador del potrero.
- Al cargar, el texto persistido se resuelve contra las reglas registradas; se elimina `Enum.Parse` pero no cambia el contenido de los archivos.
- Los rangos de edad, clases concretas creadas, excepciones, mensajes y eventos permanecen idénticos.

Por qué esa frontera

- `IReglasTipoPotrero`: ya era la abstracción estable introducida por H-04; agregar otra abstracción paralela duplicaría la frontera.
- `TipoPotrero` en la implementación: permite que una categoría nueva se identifique con un concepto del negocio sin modificar una lista central dentro de Potrero.
- `string` en la frontera web/persistencia: preserva exactamente los valores textuales que el sistema ya muestra y almacena.

- Program: es el lugar permitido para conocer concretos y ampliar la composición; Potrero permanece cerrado a modificación.

Qué se gana

- Potrero deja de contener un catálogo cerrado de categorías.
- Agregar una nueva categoría no modifica `anadir_res()`, el constructor lógico de Potrero ni las reglas existentes.
- No existen switch ni cadenas de if equivalentes.
- La misma abstracción de H-04 sirve para DIP y OCP sin crear mecanismos paralelos.
- Las categorías pueden crecer durante los próximos años mediante nuevas implementaciones y configuración.

H-06 — Potrero combina incorporación, reglas específicas y coordinación de eventos

Problemas señalados en el hallazgo

Potrero mezcla la incorporación de reses con reglas específicas por categoría y con la coordinación de eventos de potrero y peso.

`Potrero.anadir_res()` concentraba tres razones de cambio: la administración/incorporación de reses, las reglas específicas de edad/construcción de cada categoría y la coordinación de cuatro publishers. El hallazgo señala precisamente que cambios en reglas de edad, creación de animales o eventos afectan la misma clase.

En el flujo original de incorporación, Potrero concentraba una secuencia extensa de decisiones y efectos: validaba la entrada y la capacidad, determinaba el rango de edad, construía el subtipo de res, lo agregaba a la colección, preparaba y disparaba cuatro eventos en un orden específico, componía los mensajes obtenidos y devolvía el resultado final. La mezcla de estas tareas evidencia que reglas, creación y coordinación de notificaciones cambiaban dentro de la misma operación.

Las validaciones específicas de edad y la construcción concreta ya no pertenecen a Potrero.

`IReglasTipoPotrero` y sus implementaciones contienen esa variación sin switch ni enum. H-06 no duplica esa solución; se concentra en la responsabilidad que todavía permanece mezclada: coordinación de eventos.

Solución aprobada

Cambio detallado

Responsabilidad que permanece en Potrero

- Conservar `Identificacion`, `L_reses` y la regla de tipo recibida.
- Validar parámetros generales de incorporación, incluido nombre y capacidad máxima del propio potrero.
- Agregar la Res creada por `IReglasTipoPotrero`.
- Construir el mismo mensaje final y mantener la misma envoltura de excepciones de `anadir_res()`.

Nueva frontera SRP

`INotificacionIncorporacionRes` define una única capacidad: obtener los mensajes generados después de incorporar una res a un potrero. `NotificacionIncorporacionRes` implementa esa capacidad y recibe por construcción las abstracciones de los cuatro notificadores que necesita. La clase no crea publishers ni decide reglas de incorporación; únicamente coordina la secuencia de notificaciones posteriores.

NotificacionIncorporacionRes concentra exclusivamente la secuencia y composición de notificaciones posteriores a la incorporación. Debe invocar los cuatro notificadores en el mismo orden del código original y concatenar únicamente los mensajes no vacíos con los mismos saltos de línea.

No se aceptan new de publishers en la nueva clase

La primera alternativa de crear PublisherPesoMin, PublisherPesoVenta, PublisherPotreroMitad y PublisherPotreroLleno dentro de NotificacionIncorporacionRes fue descartada. Extraer una responsabilidad y volverla dependiente de cuatro concretos introduciría una nueva rigidez. Por ello se crean cuatro contratos estrechos.

- INotificadorPesoMin
- INotificadorPesoVenta
- INotificadorPotreroMitad
- INotificadorPotreroLleno

NotificacionIncorporacionRes depende de cuatro contratos estrechos: notificación de potrero a mitad de capacidad, potrero lleno, peso mínimo y peso de venta. Los publishers existentes proporcionan esas capacidades mediante las interfaces correspondientes. Así, la coordinación general conoce abstracciones y cada publisher conserva su responsabilidad específica.

Cómo se preservan los publishers existentes

Los publishers no se sustituyen ni se reescriben en H-06. Conservan sus delegates, events e Informar_* originales. Cada uno incorpora de manera aditiva Notificar(...): string, que captura temporalmente el mensaje generado por su evento, invoca el mismo Informar_* y devuelve ese mensaje o cadena vacía. La implementación no debe interceptar ni cambiar las excepciones del Informar_*; si ocurre un error, debe propagarse hasta el catch original de anadir_res().

Al implementar Notificar, el manejador temporal debe retirarse después de la invocación para no acumular nuevas suscripciones. Esto no cambia el mensaje observable del flujo actual y evita trasladar al nuevo diseño una acumulación de closures que no representa estado de negocio.

Potrero después de H-06

Después de H-06, Potrero conserva las reglas de tipo y una referencia a INotificacionIncorporacionRes. La operación de incorporar una res mantiene las validaciones generales, utiliza IReglasTipoPotrero para validar y crear la entidad, incorpora la nueva Res a la colección y delega la obtención de mensajes en la abstracción de notificación. El mensaje final y la envoltura de excepciones permanecen iguales; solo cambia la distribución interna de responsabilidades.

Por qué esa frontera

- Potrero: cambia cuando cambia cómo administra su propio estado o el flujo general de incorporación.
- IReglasTipoPotrero: cambia/extiende cuando cambian reglas específicas de edad o construcción de una categoría.
- NotificacionIncorporacionRes: cambia cuando cambia la secuencia/composición de notificaciones posteriores a incorporar una res.
- Publishers: conservan sus responsabilidades particulares y podrán evolucionar en H-25/H-26 sin volver a acoplar Potrero.

Aplicación de DIP de apoyo

Aunque H-06 es formalmente un hallazgo SRP, la nueva clase no debe introducir una violación DIP. NotificacionIncorporacionRes depende de interfaces específicas y Program, como composition root, conoce

los publishers concretos y registra qué implementación satisface cada contrato. PotreroService recibe INotificacionIncorporacionRes y la suministra al construir el Potrero.

Matriz explícita de inversión de dependencias

Campo exigido	H-06
Módulo de alto nivel	Potrero respecto a la coordinación global; NotificacionIncorporacionRes respecto a las notificaciones particulares.
Módulo de bajo nivel / detalle concreto	NotificacionIncorporacionRes y los publishers PublisherPotreroMitad, PublisherPotreroLleno, PublisherPesoMin y PublisherPesoVenta.
Abstracción que desacopla	INotificacionIncorporacionRes y las cuatro interfaces INotificador... específicas.
Composition root	p_mvcHacienda/Program.cs, método Main: registra los cuatro INotificador... con sus publishers concretos y construye INotificacionIncorporacionRes; el contenedor la entrega a PotreroService.

Beneficio de negocio: cambiar la forma de notificar ya no obliga a modificar ni volver a probar las reglas de incorporación del potrero. Las pruebas se concentran en la política afectada, reduciendo tiempo de regresión y riesgo de bloquear el alta de reses por una modificación puramente informativa.

H-07 — Autenticacion concentra gestión de usuarios, autenticación y autorización

Problemas señalados en el hallazgo

Autenticacion concentra el estado de usuarios, creación, consultas, validación de credenciales y autorización, reuniendo responsabilidades de seguridad y gestión que cambian por motivos distintos.

Además realizaba IAutenticacion, aunque ese contrato únicamente exponía AutorizarOperacion(), por lo que incluso el nombre de la abstracción no coincidía con la responsabilidad que expresaba.

En el AS-IS, Autenticacion mantenía la colección de usuarios y, en la misma clase, realizaba creación, listado, búsqueda, validación de credenciales y autorización de operaciones. Además, implementaba IAutenticacion aunque ese contrato solo representaba autorización. La combinación de estado, gestión y seguridad hacía que cambios de naturaleza distinta afectaran el mismo componente.

Responsabilidad	Qué puede cambiar	Frontera elegida	Por qué no se mezcla
Estado compartido	cómo se mantiene el conjunto de usuarios	RegistroUsuarios	almacenar estado no debe decidir reglas de creación, búsqueda o seguridad
Consulta	criterios de búsqueda/listado/existencia	ConsultaUsuarios	consultar puede crecer con filtros sin afectar creación o autenticación
Creación	validaciones de alta, duplicados y creación de Usuario	CreacionUsuario	las reglas de onboarding pueden cambiar sin tocar búsquedas o permisos
Inicialización	usuarios predeterminados /	InicializacionUsuarios	el arranque puede migrar a

Responsabilidad	Qué puede cambiar	Frontera elegida	Por qué no se mezcla
	fuelle de bootstrap		configuración u otra fuente sin modificar creación normal
Autenticación	comparación de credenciales	Autenticacion	una política de credenciales cambia por motivos de seguridad, no por consultas o permisos
Autorización	permisos para ejecutar operaciones	AutorizacionOperacion	las políticas de acceso evolucionan independientemente de credenciales y gestión

Solución aprobada

Cambio detallado

Frontera TO-BE aprobada

La frontera TO-BE divide la antigua responsabilidad de Autenticacion en piezas cohesionadas. RegistroUsuarios conserva el estado compartido y permite leer o agregar usuarios; ConsultaUsuarios se ocupa de listar, buscar y comprobar existencia; CreacionUsuario realiza el alta; InicializacionUsuarios prepara los usuarios iniciales; Autenticacion valida credenciales; y AutorizacionOperacion decide si una operación está permitida. Cada clase mantiene únicamente las operaciones que corresponden a su razón de cambio.

La colección original no se duplica. RegistroUsuarios es el único dueño del estado y Program debe construir una sola instancia compartida. Esto es esencial para conservar el comportamiento: creación, autenticación, autorización y consulta deben observar el mismo conjunto de usuarios.

Por qué hay varias interfaces y no una interfaz grande

Las interfaces no se crean por cantidad de clases ni para 'hacer que parezca SOLID'. Se diseñan desde las necesidades del consumidor. Si existiera una interfaz IUserios con Agregar, ObtenerTodos, buscar, listar, crear, autenticar y autorizar, Autenticacion dependería de métodos de escritura y autorización que no utiliza; AutorizacionOperacion dependería de creación; e InicializacionUsuarios dependería de consultas. Eso volvería a introducir acoplamiento y anticiparía el mismo problema que más adelante se denuncia formalmente para interfaces demasiado amplias.

Interfaz	Consumidor principal	Capacidad exacta	Justificación
ILecturaUsuarios	ConsultaUsuarios, Autenticacion, AutorizacionOperacion	leer el conjunto	estos consumidores no deben poder depender contractualmente de Agregar
IEscrituraUsuarios	CreacionUsuario, InicializacionUsuarios	agregar Usuario	solo quienes mutan el conjunto reciben escritura
IConsultaUsuarios	CreacionUsuario y futuros clientes de consulta	buscar/listar/existencia	separa reglas de consulta del almacenamiento
ICreacionUsuario	cliente del caso de uso de alta	crear usuario	permite cambiar validaciones de alta sin exponer consultas/seguiridad
IInicializacionUsuarios	Program / arranque	inicializar usuarios	el bootstrap tiene ciclo de

Interfaz	Consumidor principal	Capacidad exacta	Justificación
		predeterminados	vida y razón de cambio propios
IValidacionCredenciales	cliente de autenticación	validar credenciales	no expone autorización
IAutorizacionOperacion	cliente de permisos	autorizar una operación	reemplaza IAutenticacion, cuyo nombre no representaba su única operación

Esta cantidad de interfaces es intencional porque cada contrato representa una capacidad distinta y tiene consumidores distintos. No se divide buscar y listar en interfaces separadas porque ambas operaciones comparten la misma razón de cambio: consultar usuarios. La frontera se detiene donde deja de existir una razón de cambio independiente.

Qué cambia exactamente en cada elemento

- Autenticacion: pierde colección, crear/listar/buscar, inicialización y AutorizarOperacion; conserva únicamente ValidarCredenciales y recibe ILecturaUsuarios.
- RegistroUsuarios: recibe la colección usuarios_registrados y se vuelve el único dueño del estado compartido. No decide reglas de negocio.
- ConsultaUsuarios: recibe listar_usuarios() y buscar_usuario(); agrega ExisteUsuario como consulta interna para que creación no conozca la colección concreta.
- CreacionUsuario: recibe íntegramente crear_usuario(): mismas validaciones, creación de Usuario, mensajes y excepciones.
- InicializacionUsuarios: recibe exclusivamente la carga de los mismos usuarios/credenciales predeterminados que antes ocurría en el constructor de Autenticacion.
- AutorizacionOperacion: recibe íntegramente AutorizarOperacion(), incluidas sus comparaciones, mensajes, excepciones y condiciones actuales por rol.
- IAutenticacion: se elimina porque su única operación era de autorización; se sustituye por IAutorizacionOperacion y se crea IValidacionCredenciales para autenticación real.

Por qué esa frontera

La frontera se eligió con una prueba de razón de cambio. Dos operaciones permanecen juntas cuando un mismo actor o política las haría cambiar conjuntamente; se separan cuando pueden evolucionar sin relación entre sí. Buscar y listar permanecen juntos porque son consultas. Validar credenciales y autorizar se separan porque una política de contraseña puede cambiar sin que cambien los permisos, y un nuevo rol puede cambiar permisos sin alterar cómo se comprueba una contraseña. Inicializar usuarios tampoco es 'crear un usuario' en sentido del caso de uso: ocurre por ciclo de vida del sistema y puede cambiar de fuente independientemente.

Dependencias: por qué lectura y escritura están segregadas

RegistroUsuarios proporciona las capacidades de lectura y escritura del conjunto de usuarios. ConsultaUsuarios consume solo lectura; CreacionUsuario combina la capacidad de consulta con la de escritura; InicializacionUsuarios necesita únicamente escritura; y Autenticacion y AutorizacionOperacion consumen lectura. Esta distribución hace que cada clase reciba solo el contrato que corresponde a su responsabilidad.

La separación lectura/escritura evita que una clase que solo valida credenciales reciba una interfaz que también permite registrar usuarios. Es una decisión de ISP de apoyo. Asimismo, las clases consumen abstracciones y Program conoce los concretos; esto evita introducir nuevas dependencias rígidas mientras se corrige SRP.

Ajuste de contrato de lectura: `ILecturaUsuarios.ObtenerTodos()` y la implementación equivalente de `RegistroUsuarios` devuelven `ICollection<Usuario>`, no `List<Usuario>`. Así una clase que recibe únicamente `ILecturaUsuarios` no puede modificar contractualmente la colección mediante `Clear()`, `Add()` u otras operaciones de `List`. El estado interno puede seguir siendo `List<Usuario>`; la restricción está en la frontera expuesta al consumidor.

Composition root y preservación del único estado

Program debe resolver la gráfica de objetos con una única instancia de `RegistroUsuarios`. Esa misma instancia satisface lectura y escritura. Después se construyen `ConsultaUsuarios`, `CreacionUsuario`, `InicializacionUsuarios`, `Autenticacion` y `AutorizacionOperacion` contra esas abstracciones. `InicializacionUsuarios.Inicializar()` se ejecuta una sola vez durante el arranque, en el mismo momento lógico en que el sistema original disponía de sus usuarios predeterminados.

Program crea una sola instancia de `RegistroUsuarios` y la utiliza como fuente común para las capacidades de lectura y escritura. Sobre esa misma instancia construye `ConsultaUsuarios`, `CreacionUsuario`, `InicializacionUsuarios`, `Autenticacion` y `AutorizacionOperacion`. La inicialización de usuarios predeterminados se ejecuta una sola vez durante el arranque, preservando un único estado compartido.

Matriz explícita de inversión de dependencias

Módulo de alto nivel	Módulo de bajo nivel / detalle concreto	Abstracción que desacopla	Composition root
ConsultaUsuarios	RegistroUsuarios	ILecturaUsuarios	p_mvcHacienda/Program.cs → Main → builder.Services (misma instancia singleton cuando corresponde)
CreacionUsuario	ConsultaUsuarios	IConsultaUsuarios	p_mvcHacienda/Program.cs → Main → builder.Services (misma instancia singleton cuando corresponde)
CreacionUsuario	RegistroUsuarios	IEscrituraUsuarios	p_mvcHacienda/Program.cs → Main → builder.Services (misma instancia singleton cuando corresponde)
InicializacionUsuarios	RegistroUsuarios	IEscrituraUsuarios	p_mvcHacienda/Program.cs → Main → builder.Services (misma instancia singleton cuando corresponde)
Autenticacion	RegistroUsuarios	ILecturaUsuarios	p_mvcHacienda/Program.cs → Main → builder.Services (misma instancia singleton cuando corresponde)
AutorizacionOperacion	RegistroUsuarios	ILecturaUsuarios	p_mvcHacienda/Program.cs → Main → builder.Services

Módulo de alto nivel	Módulo de bajo nivel / detalle concreto	Abstracción que desacopla	Composition root
			(misma instancia singleton cuando corresponde)

Program debe cablear estas relaciones sobre una sola instancia de RegistroUsuarios. La inversión no cambia el estado observable: simplemente impide que las clases de alto nivel conozcan o construyan la implementación concreta.

Conducta observable que NO puede cambiar

- Se mantienen los mismos usuarios y credenciales predeterminados; solo cambia la clase que los incorpora al registro.
- crear_usuario conserva sus validaciones, comparación de duplicados, creación de Usuario, mensaje de éxito y envoltura de excepciones.
- listar_usuarios y buscar_usuario conservan firmas y semántica de retorno.
- ValidarCredenciales conserva exactamente sus comparaciones y resultado booleano; H-07 no introduce hash, nuevas políticas ni normalización adicional.
- AutorizarOperacion conserva exactamente la comprobación de usuario registrado, mensajes/excepciones y las ramas actuales por rol. H-08 las reorganizará después sin cambiar su resultado.
- Existe un único conjunto compartido de usuarios. No se crean listas independientes para autenticación, autorización o consultas.

Impacto para negocio y mantenimiento

Cambio futuro	Antes de H-07	Después de H-07	Impacto negocio
Cambiar reglas de credenciales	se modifica Autenticacion junto con gestión y permisos	se modifica Autenticacion	menos pruebas y menor riesgo de romper permisos o altas
Agregar/modificar permisos o roles	se abre la misma clase que autentica y gestiona usuarios	se limita a AutorizacionOperacion; H-08 hará la extensión abierta	reduce riesgo de alterar autenticación ya probada
Cambiar validaciones para crear usuarios	se toca la clase de seguridad completa	se modifica CreacionUsuario	menor tiempo de mantenimiento y pruebas más focalizadas
Agregar nuevos filtros/búsquedas	se toca la clase que también autentica	se modifica ConsultaUsuarios	evolución de consultas sin riesgo sobre credenciales
Cambiar usuarios iniciales o su fuente	se modifica constructor de Autenticacion	se modifica InicializacionUsuarios	permite migrar bootstrap/configuración sin tocar seguridad operacional

El beneficio principal no es 'tener más clases'. Es reducir el radio de impacto de cada cambio. Menos código no relacionado debe volver a probarse, disminuye el riesgo de regresión y el costo de mantenimiento se concentra en la política que realmente cambia.

Qué se gana

- Cohesión mayor: cada clase tiene una razón de cambio defendible.
- Acoplamiento menor: autenticación y autorización ya no poseen ni mutan directamente la colección.
- Pruebas más pequeñas: cada política puede probarse con una implementación controlada de la capacidad que consume.

- Extensibilidad: H-08 puede rediseñar permisos sin volver a intervenir autenticación, creación o consultas.
- Reutilización futura: H-20 puede aprovechar estas fronteras cuando se revise UsuarioService, sin declarar ese hallazgo resuelto anticipadamente.
- Nomenclatura más precisa: IAutorizacionOperacion reemplaza un IAutenticacion que realmente no expresaba autenticación.

Por qué no se fragmenta todavía más

No se crea una clase por cada método. Buscar, listar y comprobar existencia permanecen en ConsultaUsuarios porque pertenecen a la misma capacidad y normalmente evolucionan juntas. Tampoco se crea un constructor/fábrica de Usuario porque solo existe una entidad Usuario y no hay un eje de variación de implementaciones en H-07. La frontera busca cohesión y flexibilidad, no maximizar el número de clases.

Relación con H-08 y H-20

- H-08: la lógica que discriminaba roles fue primero aislada en AutorizacionOperacion por H-07 y posteriormente se abre a extensión mediante IReglasAutorizacionRol y reglas separadas por rol, sin volver a tocar autenticación, consultas ni creación.
- H-20: UsuarioService también mezcla gestión, autenticación, consultas, estadísticas y Claims. Las fronteras de H-07 son candidatas a reutilización, pero H-20 no se declara resuelto hasta analizar ese servicio completo y su persistencia.

H-08 — La autorización depende de roles conocidos

Problemas señalados en el hallazgo

La autorización conoce explícitamente los roles admin, empleado y visitante, de modo que un nuevo rol o un cambio de permisos obliga a modificar la lógica central.

En el AS-IS, AutorizarOperacion contenía decisiones explícitas para administrador, empleado y visitante. Cada rama aplicaba los permisos propios del rol y la aparición de un rol nuevo obligaba a modificar la misma operación central. El problema no era únicamente la cantidad de condiciones, sino que el algoritmo de autorización conocía todas las variantes existentes.

Solución aprobada

Cambio detallado

Solución: un contrato de reglas de autorización por rol

IReglasAutorizacionRol define el contrato mínimo de una política de autorización: cada implementación declara el rol que representa y puede responder si una operación determinada está permitida.

AutorizacionOperacion utiliza ese contrato sin conocer las reglas particulares de administrador, empleado o visitante.

IReglasAutorizacionRol representa exactamente el eje de variación identificado por H-08. No autentica, no busca usuarios, no administra el registro y no produce el mensaje final: solo expresa qué operaciones permite un rol. La coordinación general continúa perteneciendo a AutorizacionOperacion.

Reglas actuales separadas por razón de cambio

- ReglaAutorizacionAdministrador: Rol = "admin"; recibe literalmente la condición de permisos que pertenecía a la rama admin.

- `ReglaAutorizacionEmpleado`: Rol = "empleado"; recibe literalmente la condición de permisos que pertenecía a la rama empleado.
- `ReglaAutorizacionVisitante`: Rol = "visitante"; recibe literalmente la condición de permisos que pertenecía a la rama visitante.

Los permisos concretos no se redefinen en H-08. La implementación debe trasladar literalmente las condiciones existentes. La separación solo cambia dónde vive cada regla.

Por qué una clase por rol y no una sola clase de permisos

Una única clase `ReglasRoles` con todos los roles seguiría teniendo una razón de cambio por cada política de permisos y se convertiría en el nuevo archivo que habría que modificar cada vez que naciera un rol o cambiara uno existente. Separar las reglas permite que modificar permisos del administrador no obligue a tocar empleado, visitante ni la coordinación general. La frontera se traza por política que cambia independientemente, no por número de líneas.

Usuario.Rol: evolución compatible

El sistema actual utiliza el nombre del usuario como identificador de rol. Cambiar de inmediato persistencia o credenciales rompería la restricción de comportamiento observable. Por ello `Usuario` incorpora Rol de forma aditiva.

`Usuario` conserva el constructor histórico de nombre y contraseña, asignando el rol a partir del nombre para mantener la compatibilidad con los datos actuales. Además, dispone de una forma de construcción que permite indicar el rol explícitamente. Esta ampliación prepara la evolución futura sin cambiar la semántica de los usuarios existentes ni el formato persistido.

Hoy `admin`, `empleado` y `visitante` continúan obteniendo el mismo rol que antes, porque `Rol == Nombre`. La sobrecarga permite que en el futuro un usuario pueda llamarse de una forma distinta a su rol sin rediseñar `AutorizacionOperacion`.

Persistencia y conducta observable

- No se modifica el formato actual de los archivos de usuarios.
- Un usuario cargado con el constructor existente vuelve a obtener `Rol = Nombre`, por lo que `admin/empleado/visitante` conservan su autorización.
- Un usuario cuyo nombre no corresponda a reglas configuradas continúa sin obtener permisos de esos tres roles.
- Se conservan exactamente los mensajes y excepciones de autorización/denegación, incluida la forma actual de comunicar una autorización exitosa.
- La comprobación previa de que el usuario pertenece al registro compartido continúa intacta.

Cómo queda `AutorizacionOperacion`

`AutorizacionOperacion` conserva dos colaboradores: una capacidad de lectura de usuarios y un conjunto de reglas accesibles por rol. El procedimiento general permanece estable: primero comprueba que el usuario sea válido dentro del registro, después localiza la regla que corresponde a su rol, consulta si la operación está permitida y finalmente conserva la misma salida o excepción observable del sistema original.

La consulta de existencia de una clave no enumera variantes. Agregar un rol no exige agregar una nueva rama a `AutorizacionOperacion`; la operación permanece cerrada a modificación y abierta a nuevas implementaciones.

Por qué no usamos switch, enum ni herencia de Usuario

- Switch / if por roles: solo trasladaría el catálogo cerrado y seguiría obligando a modificar el algoritmo central.
- Enum Rol: representaría un conjunto cerrado y obligaría a editar el catálogo central cuando apareciera otra categoría.
- UsuarioAdministrador/UsuarioEmpleado/UsuarioVisitante: confundiría identidad del usuario con política de autorización, haría el rol estructuralmente rígido e introduciría herencias nuevas que no son necesarias.

Por qué esa frontera

Frontera	Razón de cambio	Qué no debe conocer
AutorizacionOperacion	cambia el procedimiento general de autorización, verificación y composición del resultado	detalles de permisos de admin/empleado/visitante
ReglaAutorizacionAdministrador	cambian permisos del administrador	reglas de otros roles
ReglaAutorizacionEmpleado	cambian permisos del empleado	reglas de otros roles
ReglaAutorizacionVisitante	cambian permisos del visitante	reglas de otros roles
Usuario.Rol	cambia la asociación usuario-rol	la lógica que decide permisos

Demostración de OCP con un rol futuro

La extensibilidad puede comprobarse con un rol futuro, por ejemplo Supervisor. Para incorporarlo se crea una nueva implementación de IReglasAutorizacionRol que declara el rol y sus permisos, y se registra en Program. AutorizacionOperacion y las reglas existentes de administrador, empleado y visitante permanecen sin modificaciones.

Ese escenario es el criterio práctico de apertura/cierre: la capacidad crece añadiendo una implementación, no editando las ramas ya probadas.

Matriz explícita de inversión de dependencias

Campo exigido	H-08
Módulo de alto nivel	AutorizacionOperacion.
Módulo de bajo nivel / detalle concreto	ReglaAutorizacionAdministrador, ReglaAutorizacionEmpleado y ReglaAutorizacionVisitante.
Abstracción que desacopla	IReglasAutorizacionRol.
Composition root	p_mvcHacienda/Program.cs, método Main: construye las ReglaAutorizacionX, forma el IReadOnlyDictionary<string, IReglasAutorizacionRol> y el contenedor lo suministra a AutorizacionOperacion.

Beneficio para negocio

Cambio futuro	Antes	Después	Beneficio
Agregar un rol	editar autorización ya	añadir una clase de reglas +	menor tiempo de

Cambio futuro	Antes	Después	Beneficio
	probada y agregar otra rama	composición	implementación y regresión
Cambiar permisos de admin	editar una operación que contiene otros roles	modificar solo ReglaAutorizacionAdministrador	menor riesgo de alterar permisos de empleado/visitante
Separar nombre y rol	rediseñar la autorización basada en Nombre	usar la sobrecarga con Rol	evolución compatible sin migración inmediata de archivos

En seguridad, reducir el radio de cambio también reduce el riesgo de conceder o denegar permisos incorrectamente. El mantenimiento se vuelve más localizado y las pruebas pueden enfocarse en la política modificada.

Qué se gana

- AutorizacionOperacion queda estable frente al crecimiento de roles.
- Cada rol tiene una política cohesionada y comprobable de manera independiente.
- No hay switch, enum ni cadena if/else por categorías.
- No se introduce ninguna herencia nueva, por lo que H-08 no aumenta la deuda de verificación LSP.
- La dependencia de alto nivel se dirige a IReglasAutorizacionRol y los concretos se conocen en Program.
- Rol desacopla gradualmente la identidad del usuario de su política de permisos sin romper datos existentes.

Bloque LSP H-09, H-10 y H-11 — contrato único de edad para la jerarquía Res

Problemas señalados en el hallazgo

Ternero, Cebon y Novillo declaran versiones propias de Edad además del contrato de Res, por lo que la asignación puede comportarse de forma distinta según el subtipo utilizado.

Solución aprobada

Cambio detallado

Frontera elegida: reglas de edad como eje de negocio independiente

Se crea un bloque ReglasEdadRes separado. No se mezclan las reglas de edad con vacunación ni se concentran todas las reglas de Ternero/Cebon/Novillo en clases monolíticas como ReglasTernero. La edad, la vacunación y la incorporación a potrero pueden cambiar por motivos distintos; por ello son ejes de política diferentes.

Reglas/

ReglasEdadRes/
 ReglaEdadTernero
 ReglaEdadCebon
 ReglaEdadNovillo

Interfaces/

IReglaEdadRes

Contrato de edad

IReglaEdadRes representa la política mínima necesaria para determinar si una edad es válida para una res. El contrato no produce mensajes ni conoce el contexto de Potrero; únicamente responde la validez, permitiendo que cada consumidor conserve sus propios mensajes y excepciones observables.

La interfaz devuelve únicamente validez. Esta decisión es importante para preservar comportamiento: el mismo rango se reutiliza en contextos diferentes, pero cada contexto conserva su propio mensaje/excepción observable. Potrero puede seguir produciendo su mensaje contextual de incorporación y Res.Edad conserva el rechazo que tenía la asignación directa; la política común no impone un mensaje nuevo.

Reglas concretas y valores preservados

Regla	Rango actual	Responsabilidad	Qué no contiene
ReglaEdadTernero	0–12	determinar si la edad es válida para Ternero	vacunación, creación, mensajes de Potrero
ReglaEdadCebon	13–48	determinar si la edad es válida para Cebon	vacunación, creación, mensajes de Potrero
ReglaEdadNovillo	≥ 49	determinar si la edad es válida para Novillo	vacunación, creación, mensajes de Potrero

Los límites de edad salen de ReglaRes. ReglaRes conserva las reglas de peso, porque peso y edad son políticas con razones de cambio distintas. Así se evita que ReglaRes continúe acumulando reglas heterogéneas.

Un solo miembro Edad en Res

Res se convierte en la única propietaria del miembro Edad y mantiene internamente la política IReglaEdadRes que protege ese estado. Ternero, Cebon y Novillo continúan heredando de Res, pero ya no declaran una versión propia de Edad. Esto asegura que cualquier consumidor, independientemente del subtipo concreto, utilice exactamente el mismo contrato público para consultar o asignar la edad.

Res recibe IReglaEdadRes durante construcción. Toda asignación pasa por el mismo miembro Res.Edad; por tanto el subtipo ya no oculta, sobrescribe ni redefine la operación que el cliente utiliza. La variación de rango se compone mediante la regla asociada.

Reutilización desde las reglas de Potrero sin duplicar rangos

ReglaTipoPotreroTernero recibe una IReglaEdadRes y la utiliza para comprobar la edad en el contexto de incorporación. Si la edad es inválida conserva el mensaje propio del potrero; si es válida, crea el Ternero suministrándole la misma política de edad. De este modo, el rango se define una sola vez y es reutilizado tanto por la entidad como por el flujo de incorporación.

La misma estructura se aplica a Cebon y Novillo. De esta manera los rangos no aparecen dos veces: una modificación futura de la edad de Novillo ocurre en ReglaEdadNovillo y tanto la entidad como el flujo de Potrero reutilizan esa política.

Por qué no colocar la edad dentro de ReglaTipoPotreroX

Aunque técnicamente ReglaTipoPotreroNovillo podría implementar también la regla de edad, esa ubicación mezclaría dos motivos de cambio: la validez intrínseca de la edad de una Res y el proceso de incorporación/construcción dentro de un Potrero. Una regla de edad también se necesita cuando la Res se reconstruye, se modifica directamente o se utiliza fuera de Potrero. Por eso la política de edad vive en ReglasEdadRes y ReglaTipoPotreroX la consume.

Por qué no crear una sola ReglasTernero / ReglasCebon / ReglasNovillo

Agrupar edad, vacunación, peso y cualquier regla futura por categoría volvería a crear clases con razones de cambio heterogéneas. Una norma sanitaria podría cambiar sin que cambie el rango de edad; una política de edad puede cambiar sin modificar vacunación. La arquitectura se organiza por política de negocio y luego por variante, no por un contenedor genérico de todo lo relacionado con una categoría.

Composition root y creación

Program conoce los concretos y configura los pares correspondientes: ReglaEdadTernero se suministra a ReglaTipoPotreroTernero; lo mismo para Cebon y Novillo. Los constructores de las reses reciben IReglaEdadRes y la entregan a Res. No se crea ReglaEdadTernero dentro de Ternero, no hay switch, enum, selector ni factoría.

Program compone cada categoría con la política de edad y la política de tipo de potrero que le corresponden. La regla de edad de Ternero se comparte con la regla que construye Terneros; la misma relación se mantiene para Cebon y Novillo. Todo el enlace se expresa mediante IReglaEdadRes, evitando que las entidades creen o seleccionen internamente sus reglas concretas.

Preservación de persistencia y otros flujos

El cambio de constructor no puede dejar la carga sin resolver. La parte de PersistenciaService que cargaba Ternero/Cebon/Novillo reutiliza IReglasTipoPotrero.CrearRes para suministrar la regla de edad correcta. Esta decisión se completa posteriormente en H-14, donde también se extrae la carga de Bacteriana/Viva. Los textos almacenados y los valores de edad no cambian.

Qué se gana e impacto para negocio y mantenimiento

Cambio / riesgo	AS-IS	TO-BE	Beneficio
Cambiar rango de Novillo	regla repartida entre subtipo y flujo de Potrero	modificar ReglaEdadNovillo	menos archivos/pruebas afectados
Uso polimórfico como Res	puede ejecutar una propiedad Edad propia del subtipo	siempre usa Res.Edad	menor riesgo de fallos sorpresa
Diagnóstico de edad inválida	hay que identificar qué setter concreto intervino	la política de edad está explícita y localizada	menor tiempo de diagnóstico
Agregar nueva categoría	repetir lógica de edad en varias ubicaciones	agregar ReglaEdadNueva + composición	crecimiento aditivo y menor regresión

El valor para el negocio es reducir errores que aparecen únicamente en ejecución y el tiempo de diagnóstico/corrección señalado por los hallazgos. El valor técnico es un contrato polimórfico único, reglas reutilizables y menor duplicación.

H-09 — Novillo → Res: justificación de la herencia conservada

Verificación Fase 3	Resultado TO-BE
Por qué herencia	Novillo sigue siendo una Res y participa como tal en colecciones, ventas, vacunación y operaciones comunes.
Por qué no composición	Componer una Res duplicaría identidad y exigiría delegar Nombre, Peso, Edad y vacunas; el defecto era la redefinición de Edad.
Precondiciones	Novillo no redefine Edad ni fortalece su setter; usa el mismo contrato heredado parametrizado por IReglaEdadRes.

Verificación Fase 3	Resultado TO-BE
Postcondiciones	Si la asignación finaliza correctamente, Edad == valor solicitado.
Invariantes	La edad se mantiene válida conforme a ReglaEdadNovillo (rango actual ≥ 49).
Excepciones	La edad inválida conserva el rechazo observable original; no aparece una excepción sorpresa por ejecutar otro miembro Edad.
Sustituibilidad	Un Novillo tratado como Res utiliza exactamente Res.Edad; la propiedad propia de Novillo desaparece.

H-10 — Ternero → Res: justificación de la herencia conservada

Verificación Fase 3	Resultado TO-BE
Por qué herencia	Ternero es una Res y reutiliza el mismo estado y operaciones de la entidad base.
Por qué no composición	Composición agregaría delegación/duplicación de la entidad sin resolver mejor el problema del contrato de Edad.
Precondiciones	Ternero no redefine Edad; usa Res.Edad con IReglaEdadRes.
Postcondiciones	Si la asignación finaliza correctamente, Edad == valor solicitado.
Invariantes	La edad se mantiene válida conforme a ReglaEdadTernero (rango actual 0–12).
Excepciones	Se conserva el rechazo observable de una edad inválida, pero ya no proviene de una versión propia de Edad.
Sustituibilidad	Un Ternero tratado como Res utiliza el mismo miembro y flujo de asignación.

H-11 — Cebon → Res: justificación de la herencia conservada

Verificación Fase 3	Resultado TO-BE
Por qué herencia	Cebon mantiene una relación es-una con Res y comparte identidad, peso, edad y vacunas aplicadas.
Por qué no composición	Crear una Res contenida obligaría a delegar toda la entidad; la herencia es válida una vez eliminado el contrato Edad incompatible.
Precondiciones	Cebon no redefine Edad ni introduce un setter alternativo.
Postcondiciones	Si la asignación finaliza correctamente, Edad == valor solicitado.
Invariantes	La edad se mantiene válida conforme a ReglaEdadCebon (rango actual 13–48).
Excepciones	El rechazo de una edad inválida se conserva sin NotImplemented ni comportamiento de un miembro oculto.
Sustituibilidad	Un Cebon puede utilizarse como Res y la asignación pasa por el mismo contrato base.

H-12 — IValidarInformacion obliga a depender de validaciones que no se usan

Problemas señalados en el hallazgo

IValidarInformacion obliga a cada validador a exponer validaciones de Res, Potrero, Vacuna y Venta aunque cada implementación real solo soporta una de esas capacidades.

El contrato original IValidarInformacion agrupaba validaciones de Res, Potrero, Vacuna y Venta, y la superclase Validacion trasladaba ese conjunto completo a todos los validadores. Como cada validador concreto solo soportaba realmente uno de esos dominios, las otras operaciones quedaban sin una implementación válida. Esta estructura es la causa del problema de ISP y LSP que se elimina en el TO-BE.

Solución aprobada

Cambio detallado

Frontera elegida: una interfaz general, pero genérica

IValidador<T> expresa una sola capacidad: validar un elemento del tipo indicado. El parámetro genérico hace que cada implementación prometa únicamente la validación que realmente soporta, evitando un contrato amplio con operaciones de dominios ajenos.

Se conserva una única idea general de validación, pero el parámetro de tipo limita el contrato al dominio real del validador. IValidador<Res> no promete validar Potrero, Vacuna ni Venta; IValidador<Vacuna> tampoco conoce los demás tipos. La generalidad está en la operación Validar, no en una lista de métodos heterogéneos.

Por qué IValidador<T> y no cuatro interfaces IValidadorRes/IValidadorPotrero/...

Cuatro interfaces independientes resolverían el síntoma inmediato, pero repetirían el mismo contrato conceptual cuatro veces. IValidador<T> expresa que todas esas clases tienen una capacidad común — validar un elemento— sin obligarlas a validar tipos ajenos. Además, permite que en el futuro existan varias validaciones distintas del mismo tipo sin ampliar ninguna interfaz.

La misma abstracción IValidador<Res> puede ser implementada por la validación actual de Res y, si el dominio lo requiere en el futuro, por validadores más específicos de peso, identificación u otras políticas. El contrato no necesita ampliarse: cada nueva regla se incorpora como otra implementación de la misma capacidad sobre Res.

Por qué no se crea una clase abstracta Validacion<T>

Una clase abstracta solo sería justificable si existiera estado o algoritmo común real que debiera heredarse. En el diseño actual, la única coincidencia es la firma Validar(T). Introducir herencia únicamente para compartir una firma agregaría otra jerarquía que luego tendría que justificarse frente a LSP sin aportar comportamiento reutilizable. Por eso la capacidad común se expresa con interfaz y no con herencia.

Qué sucede con los validadores actuales

Clase actual	Contrato TO-BE	Operación incompatible eliminada	Validación efectiva preservada
ValidadorRes	IValidador<Res>	ValidarPotrero / ValidarVacuna / ValidarVenta	ValidarRes

Clase actual	Contrato TO-BE	Operación incompatible eliminada	Validación efectiva preservada
ValidadorPotrero	IValidador<Potrero>	ValidarRes / ValidarVacuna / ValidarVenta	ValidarPotrero
ValidadorVacuna	IValidador<Vacuna>	ValidarRes / ValidarPotrero / ValidarVenta	ValidarVacuna
ValidadorVenta	IValidador<Venta>	ValidarRes / ValidarPotrero / ValidarVacuna	ValidarVenta

La operación existente que sí contiene la validación se conserva para compatibilidad y no se reescribe. Cada clase agrega Validar(T), que delega en esa operación existente. De esta forma el nuevo contrato puede usarse sin duplicar reglas y sin alterar el resultado, las excepciones o la lógica efectiva de validación.

ValidadorRes realiza IValidador<Res> y mantiene la operación que ya contiene la lógica de validación existente. La operación del contrato genérico se limita a delegar en esa validación, de modo que se adopta la nueva abstracción sin duplicar ni alterar las reglas actuales.

Por qué los nombres actuales no se vuelven artificialmente más específicos

La preocupación de crecimiento es válida: una clase llamada ValidadorRes podría convertirse en un contenedor de muchas reglas sin relación. Sin embargo, el material fuente disponible para H-12 solo permite afirmar que cada clase tiene una única operación de validación real; no documenta con suficiente precisión si esa operación valida peso, identificación, integridad, vigencia u otro concepto específico. Renombrarla ahora como ValidadorPesoRes o ValidadorIntegridadRes sería inventar semántica que los hallazgos no soportan.

La decisión de diseño es entonces deliberada: conservar el nombre actual para la implementación existente y establecer la regla de evolución. Cuando aparezca una segunda razón de validación claramente distinta, se crea una implementación con nombre de negocio específico que también realiza IValidador<T>; no se agregan más métodos al contrato ni se convierte ValidadorRes en una clase gigante.

Regla para el crecimiento futuro

Situación futura	Decisión	Qué se evita
Cambian varias condiciones que pertenecen a la misma razón de negocio	permanecen en el mismo validador	una clase por cada if
Aparece una validación que cambia por otra política/actor	se agrega otro IValidador<T> concreto con nombre específico	una clase monolítica por entidad
Aparece otro tipo de entidad validable	se implementa IValidador<NuevoTipo>	ensanchar IValidador<T> con otro método

Eliminación de la superclase Validacion

En el AS-IS, ValidadorRes heredaba de Validacion y quedaba obligado a convivir con operaciones para Potrero, Vacuna y Venta que no podía cumplir. En el TO-BE se elimina esa herencia y ValidadorRes realiza únicamente IValidador<Res>. Se conserva la validación efectiva de Res y desaparecen las capacidades ajenas que provocaban la violación de sustituibilidad.

Validacion no aporta un algoritmo o estado común que justifique la herencia; su efecto principal es prometer cuatro operaciones que los subtipos no pueden cumplir. Por ello la jerarquía se elimina en vez de intentar repararla con más métodos abstractos.

Relación con H-28 (LSP)

H-28 identifica que los validadores no pueden sustituir a Validacion porque tres de las cuatro operaciones terminan en NotImplementedException. H-12 elimina precisamente la causa estructural: desaparece Validacion, desaparecen las cuatro generalizaciones y cada clase promete solo una capacidad que puede cumplir. Por ello H-28 queda pre-resuelto estructuralmente. Cuando se llegue formalmente a H-28, la justificación registrará esta reutilización en lugar de crear otra arquitectura.

¿Qué pasa con PersistenciaService y los proxies?

H-12 no aprovecha la oportunidad para resolver problemas que pertenecen a otros hallazgos. PersistenciaService mantiene por ahora sus referencias y creación concreta de ValidadorRes, ValidadorPotrero, ValidadorVacuna y ValidadorVenta/proxies. Cambiar quién crea esos objetos y hacer que PersistenciaService dependa de abstracciones corresponde a H-13/H-15. Esta separación evita declarar DIP resuelto de forma anticipada.

Los proxies pueden seguir envolviendo las clases concretas; el nuevo método Validar(T) delega a la misma operación que ya contiene la lógica. La interceptación y cualquier conducta técnica asociada deben conservarse hasta que el hallazgo de persistencia/proxy sea tratado explícitamente.

Beneficio para negocio

Cambio	AS-IS	TO-BE	Impacto
Modificar validación de Res	contrato y jerarquía incluyen otros tres dominios	la implementación se concentra en IValidador<Res>	menor tiempo de prueba y menor riesgo de tocar contratos ajenos
Crear un doble de prueba para vacunas	debe convivir con un contrato de cuatro operaciones	implementa solo Validar(Vacuna)	menos código de prueba y menor costo de mantenimiento
Agregar otra validación de Res	riesgo de seguir creciendo ValidadorRes o ensanchar interfaz	se agrega otra implementación IValidador<Res>	crecimiento localizado y mayor reutilización
Reutilizar un validador	el consumidor necesita conocer qué métodos realmente soporta	el tipo genérico expresa la capacidad válida	menor riesgo de fallos en ejecución

Qué se gana

- El contrato tiene una sola responsabilidad y no obliga a implementar operaciones ajenas.
- Se elimina una jerarquía que no superaba sustituibilidad.
- Los validadores pueden crecer por extensión: varias implementaciones pueden validar el mismo T con razones de cambio diferentes.
- Las pruebas pueden usar implementaciones mínimas de IValidador<T>.
- No se introduce una clase abstracta artificial ni una nueva herencia que deba justificarse.
- No se cambia ninguna regla de validación existente; solo la forma estructural en que se expone.

H-13 — PersistenciaService mezcla persistencia, validación/proxies y HttpContext

Problemas señalados en el hallazgo

PersistenciaService mezcla serialización y acceso a archivos con creación de proxies, manejo del interceptor, lectura de HttpContext y validación de datos de Usuario.

En el AS-IS, `PersistenciaService` reunía acceso al directorio de datos, serialización y reconstrucción del estado, creación de interceptores y proxies, inicialización de esos componentes, lectura del resultado de validación desde `HttpContext` y comprobación directa de los datos obligatorios de `Usuario`. Estas responsabilidades técnicas y de validación tenían razones de cambio diferentes y justificaban su separación.

Solución aprobada

Cambio detallado

Frontera TO-BE

La separación de H-13 asigna una responsabilidad clara a cada pieza. `PersistenciaService` conserva la serialización, carga y guardado lógico; `ValidadoresPersistenciaInterceptados` administra la creación diferida de interceptores y proxies; `ResultadoValidacionHttpContext` obtiene el resultado de validación desde el contexto web; y `ValidadorDatosRequeridosUsuario` comprueba únicamente los datos obligatorios de `Usuario`.

Por qué no se crean seis clases de persistencia

No existe evidencia en H-13 de que `Potrero`, `Res`, `Vacuna`, `Venta`, `Usuario` y `VacunasAplicadas` deban evolucionar como seis mecanismos de almacenamiento independientes. Dividir por archivo produciría fragmentación sin una razón de cambio demostrada. La frontera se detiene en aquello que el hallazgo sí distingue: persistencia/serialización frente a infraestructura de validación y frente a integración HTTP.

ValidadoresPersistenciaInterceptados

`ValidadoresPersistenciaInterceptados` concentra el contexto necesario para construir y reutilizar los validadores interceptados de `Potrero`, `Res`, `Vacuna` y `Venta`. Conserva internamente el interceptor y las referencias a los proxies, y expone hacia `PersistenciaService` únicamente las capacidades `IValidador<T>` correspondientes. La inicialización sigue siendo diferida para respetar el mismo ciclo de vida del sistema original.

Esta clase recibe íntegramente la responsabilidad que antes cumplía `InicializarProxies()`. Su objetivo no es decidir reglas de negocio, sino proporcionar los mismos validadores interceptados que `PersistenciaService` necesita.

Por qué se conserva la inicialización lazy

El sistema original no crea necesariamente los proxies al construir `PersistenciaService`; `InicializarProxies()` difiere esa creación hasta el momento de uso, cuando el contexto HTTP está disponible. H-13 no cambia ese ciclo de vida. El comportamiento se traslada a `ValidadoresPersistenciaInterceptados.Inicializar()`: la primera solicitud de un validador crea interceptor y proxies; las solicitudes posteriores reutilizan las mismas instancias.

Construir todos los proxies inmediatamente en `Program` habría simplificado el código, pero sería un cambio de ciclo de vida que no es necesario para resolver SRP y podría modificar el momento en que `IHttpContextAccessor` es utilizado.

Reutilización de `IValidador<T>`

H-12 ya definió el contrato `IValidador<T>`. H-13 lo reutiliza en lugar de crear interfaces nuevas como `IValidadorPersistenciaRes` o `IValidadorPersistenciaVacuna`. Los proxies siguen envolviendo las clases existentes, pero el consumidor recibe únicamente la capacidad `Validar(T)` correspondiente a cada tipo.

ResultadoValidacionHttpContext

ResultadoValidacionHttpContext encapsula la forma actual de recuperar el resultado de validación desde el contexto HTTP. La clase consulta la misma entrada utilizada por el sistema original y, cuando no existe un valor disponible, devuelve el mensaje alternativo recibido. PersistenciaService deja así de conocer el mecanismo concreto de HttpContext.

La clave ResultadoValidacion, el acceso mediante HttpContext.Items, ToString() y el operador de fallback permanecen iguales. PersistenciaService deja de conocer IHttpContextAccessor y solicita solamente el texto final que necesita.

Por qué ResultadoValidacionHttpContext no se mezcla con los proxies

El mecanismo de creación/intercepción y el mecanismo para transportar el resultado a través de HttpContext son dos razones técnicas diferentes. Castle podría cambiar sin que cambie la ubicación del mensaje; igualmente, el mensaje podría dejar de almacenarse en HttpContext.Items sin alterar cómo se generan los proxies. Mantenerlos separados reduce el radio de cambio.

ValidadorDatosRequeridosUsuario

ValidadorDatosRequeridosUsuario implementa la capacidad de validar un Usuario y se limita a comprobar que Nombre y Contraseña contengan información. El mensaje observable continúa siendo responsabilidad del flujo de guardado; el validador solo entrega el resultado verdadero o falso de esa regla.

GuardarUsuarios contenía una validación que no pertenece a la serialización: Nombre y Contraseña son obligatorios. El nombre de la nueva clase es específico porque el código permite identificar exactamente qué datos verifica. No se crea un ValidadorUsuario genérico que pueda crecer indiscriminadamente.

El mensaje "Error: Usuario debe tener nombre y contraseña" permanece en GuardarUsuarios. El validador responde únicamente verdadero/falso, de modo que extraer la regla no cambia el texto observable ni introduce el interceptor donde antes no existía.

Qué cambia exactamente en PersistenciaService

Elemento AS-IS	Decisión H-13	Destino / estado
_httpContextAccessor	sale de PersistenciaService	ValidadoresPersistenciaInterceptados y ResultadoValidacionHttpContext
_interceptorValidacion	sale	ValidadoresPersistenciaInterceptados
cuatro _validador...Proxy	salen	ValidadoresPersistenciaInterceptados
InicializarProxies()	sale íntegramente	ValidadoresPersistenciaInterceptados.Inicializar()
ProxyGenerator	deja de ser conocido por PersistenciaService	permanece como detalle interno de la nueva frontera
HttpContext.Items["ResultadoValidacion"]	sale	ResultadoValidacionHttpContext
regla Nombre/Contraseña de Usuario	sale	ValidadorDatosRequeridosUsuario : IValidador<Usuario>
Guardar*/Cargar*	permanecen	PersistenciaService
serialización y formatos de líneas	permanecen	PersistenciaService
File.ReadAllLines/WriteAllLines y directorio	permanecen hasta H-15	PersistenciaService
reconstrucción concreta de variantes	permanece hasta H-14	PersistenciaService

Preservación estricta del comportamiento

- Las firmas públicas GuardarPotreros/Reses/Ventas/Vacunas/VacunasAplicadas/Usuarios y Cargar... no cambian.
- Los nombres de archivos, delimitadores, orden de campos y serialización textual no cambian.
- Los validadores existentes conservan sus reglas y continúan pasando por el mismo mecanismo de interceptación.
- La creación de interceptor/proxies continúa siendo lazy.
- La clave HttpContext.Items["ResultadoValidacion"] y el uso del mismo mensaje alternativo no cambian.
- GuardarUsuarios conserva literalmente "Error: Usuario debe tener nombre y contraseña".
- Los try/catch, tipos/textos de excepciones y orden observable de las operaciones deben trasladarse sin modificación.

Beneficio para negocio

Cambio futuro	Antes	Después	Impacto negocio
Cambiar mecanismo de proxy/intercepción	se modifica PersistenciaService, junto a serialización y archivos	se concentra en ValidadoresPersistenciaInterceptados	menos regresión sobre guardado/carga
Cambiar cómo se obtiene el mensaje de validación	se toca la clase que también escribe archivos	se modifica ResultadoValidacionHttpContext	menos pruebas y menor riesgo de afectar datos
Cambiar datos obligatorios de Usuario	se modifica GuardarUsuarios	se modifica ValidadorDatosRequeridosUsuario	regla de negocio localizada y menor tiempo de cambio
Cambiar formato/almacenamiento	se modifica una clase con validación/HTTP mezclados	se concentra en PersistenciaService	diagnóstico y pruebas más focalizados

Qué se gana

- PersistenciaService deja de conocer Castle, InterceptorValidarInformacion e IHttpContextAccessor.
- Cada nueva clase tiene una razón de cambio defendible y limitada.
- Se reutiliza IValidador<T> en vez de crear un segundo sistema de validación.
- La validación de Usuario queda nombrada por la regla que realmente aplica.
- H-14 resuelve posteriormente OCP de carga por tipos sin mezclar infraestructura de validación.
- H-15 trata posteriormente DIP sobre las fronteras ya separadas, evitando invertir una clase monolítica.

H-14 — PersistenciaService carga subtipos concretos al leer archivos

Problemas señalados en el hallazgo

PersistenciaService decide y construye directamente subtipos de Res y Vacuna al leer archivos, por lo que incorporar una nueva variante obliga a modificar la carga central.

Solución aprobada

Cambio detallado

Idea del cambio en una frase

PersistenciaService debe saber leer los datos, pero no debe saber construir cada subtipo. Para Res reutiliza las reglas de tipo que ya existen; para Vacuna delega la carga a CargaVacuna, y esta usa la regla específica del tipo encontrado en los datos.

Antes de H-14, PersistenciaService decidía directamente qué subtipo de Res y qué subtipo de Vacuna debía reconstruir. En el TO-BE, la carga de reses reutiliza IReglasTipoPotrero y la carga de vacunas se delega a ICargaVacuna. PersistenciaService sigue leyendo los datos, pero deja de conocer los constructores concretos de cada variante.

Carga de Res: reutilizar lo que ya existe

H-04/H-05 ya crearon IReglasTipoPotrero y una regla por categoría. H-14 no crea ICargaRes ni otro conjunto de clases, porque sería duplicar el mismo eje de variación. El texto leído (ternero, cebon o novillo) permite obtener la regla configurada y la regla ejecuta CrearRes().

```
string tipo = /* mismo valor leído actualmente */;  
IReglasTipoPotrero regla = reglas_res_por_tipo[tipo];  
Res res = regla.CrearRes(nombre, peso, edad);
```

PersistenciaService ya no contiene decisiones del tipo “si es ternero crea Ternero”. El conocimiento del constructor concreto permanece en ReglaTipoPotreroTernero/Cebon/Novillo, que ya era la frontera aprobada.

Carga de Vacuna: dos niveles con responsabilidades distintas

Elemento	Responsabilidad	Qué NO conoce
PersistenciaService	lee los datos almacenados y solicita cargar una vacuna	Bacteriana/Viva y sus constructores
ICargaVacuna / CargaVacuna	coordina la carga de una vacuna y obtiene la regla correspondiente	cómo se interpreta cada tipo concreto
IReglaCargaTipoVacuna	contrato común para una regla capaz de cargar un tipo de vacuna	otros tipos de vacuna
ReglaCargaBacteriana	interpreta los campos actuales de Bacteriana y crea Bacteriana	cómo se carga Viva
ReglaCargaViva	interpreta los campos actuales de Viva y crea Viva	cómo se carga Bacteriana

La carga de vacunas se organiza en dos niveles. PersistenciaService solicita la creación de una vacuna mediante ICargaVacuna; CargaVacuna coordina el proceso y utiliza IReglaCargaTipoVacuna para delegar la interpretación de cada variante. ReglaCargaBacteriana y ReglaCargaViva son las implementaciones que conocen los detalles particulares de sus respectivos tipos.

Contratos aprobados

ICargaVacuna expresa la capacidad de reconstruir una vacuna a partir de los campos ya leídos.

IReglaCargaTipoVacuna representa la regla específica de una variante: identifica el tipo que atiende y sabe interpretar esos campos para producir la Vacuna correspondiente. De esta forma, la coordinación y el detalle de cada subtipo permanecen separados.

TipoVacuna no es un nuevo código técnico ni un enum. Es el texto que ya permite reconocer el tipo durante la lectura. Se utiliza solo mientras todavía existen datos y aún no existe el objeto. Una vez creada la Vacuna, el sistema vuelve a trabajar con el objeto y sus abstracciones, no con ese texto para decidir comportamiento.

Por qué la interfaz recibe campos y no datoEspecifico

Se descartó la propuesta datoEspecifico porque Bacteriana y Viva no comparten semánticamente ese dato: en una variante representa un período de aplicación y en la otra una atenuación. Forzar un parámetro común ocultaría diferencias reales. Lo verdaderamente común es que ambas reglas reciben los campos que ya fueron leídos y cada una interpreta únicamente los que necesita.

ReglaCargaBacteriana interpreta exclusivamente los campos que corresponden a fechas y período de aplicación y reconstruye una Bacteriana con los mismos valores del sistema original. ReglaCargaViva interpreta los datos propios de fechas y atenuación y reconstruye una Viva. Cada regla conoce solo su variante y conserva las conversiones actuales.

Ubicación de las reglas

Las clases se organizan según su responsabilidad: las reglas específicas de carga de vacunas se ubican junto con las demás reglas de negocio; ICargaVacuna e IReglaCargaTipoVacuna se ubican entre los contratos; y CargaVacuna permanece en Servicios porque coordina la selección de la regla sin contener el detalle de cada subtipo.

Las reglas van con las demás reglas porque contienen la diferencia entre variantes. CargaVacuna permanece en Servicios porque su responsabilidad es coordinar, no definir las particularidades de un subtipo.

Por qué durante la carga se necesita TipoVacuna

Durante la lectura todavía no existe una instancia Vacuna que pueda analizarse con polimorfismo. Solo existen campos del archivo. Por eso se usa el identificador textual que ya viene en esos datos para encontrar la regla correspondiente. Intentar deducir el tipo por la forma de los campos sería frágil y volvería a introducir condiciones en CargaVacuna.

datos: "Bacteriana | ..."

↓

TipoVacuna = "Bacteriana"

↓

ReglaCargaBacteriana

↓

Vacuna ya creada

↓

NO se vuelve a usar TipoVacuna para decidir comportamiento

Por qué no usar VacunaService.CrearVacuna()

Crear una vacuna por solicitud del usuario y cargar una vacuna que ya estaba almacenada son flujos distintos. VacunaService puede contener validaciones, inventario, mensajes o eventos propios de una creación de negocio. Ejecutarlos otra vez al recuperar datos podría cambiar la conducta. Por eso la carga tiene su propia frontera y conserva únicamente las conversiones/constructores que el código actual de carga ya ejecuta.

Demostración de OCP

La apertura a extensión se comprueba con una vacuna futura, por ejemplo Recombinante. Para soportarla bastaría incorporar una nueva regla de carga que implemente IReglaCargaTipoVacuna y registrarla en

Program. PersistenciaService, CargaVacuna y las reglas actuales de Bacteriana y Viva no tendrían que modificarse, lo que demuestra que el algoritmo central queda cerrado frente a nuevas variantes.

El algoritmo estable permanece cerrado a modificación y el sistema crece agregando una nueva regla. Ese es el criterio práctico de cierre de H-14.

Qué cambia y qué se conserva

Aspecto	Cambio estructural	Conducta observable
Carga de Res	usa mapa -> IReglasTipoPotrero.CrearRes	mismos tipos, valores y objetos resultantes
Carga de Vacuna	PersistenciaService usa ICargaVacuna	mismos datos y vacunas resultantes
Bacteriana/Viva	constructores/parseos se mueven a su ReglaCarga...	mismas conversiones, fechas, excepciones y valores
Archivos	ninguno	mismo formato y contenido
Tipo de vacuna	se consulta solo para elegir la regla de carga	no se usa después como mecanismo de comportamiento

Beneficio para el negocio y el software

Cambio futuro	Antes	TO-BE	Impacto
Nueva categoría de Res	modificar carga central	nueva regla de tipo + composición	menos tiempo y regresión
Nueva categoría de Vacuna	modificar CargarVacunas/CargarVacunasAplicadas	nueva ReglaCarga... + registro en Program	menor costo de cambio
Cambiar la carga de Viva	tocar lógica compartida	solo ReglaCargaViva	menor riesgo sobre Bacteriana
Cambiar Bacteriana	riesgo de afectar ramas vecinas	solo ReglaCargaBacteriana	pruebas más focalizadas

Matriz explícita de inversión de dependencias

Campo exigido	H-14
Módulo de alto nivel	PersistenciaService para la carga general y CargaVacuna para la coordinación de vacunas.
Módulo de bajo nivel / detalle concreto	ReglaTipoPotreroTernero/Cebon/Novillo; ReglaCargaBacteriana y ReglaCargaViva.
Abstracción que desacopla	IReglasTipoPotrero, ICargaVacuna e IReglaCargaTipoVacuna.
Composition root	p_mvchacienda/Program.cs, método Main: construye ReglaCargaBacteriana/ReglaCargaViva, forma el mapa por TipoVacuna, construye CargaVacuna y registra ICargaVacuna para PersistenciaService.

H-15 — PersistenciaService depende de infraestructura concreta

Problemas señalados en el hallazgo

PersistenciaService depende de infraestructura concreta de archivos, validadores interceptados y HttpContext, dificultando sustituir almacenamiento o mecanismos técnicos sin modificar el servicio.

Solución aprobada

Cambio detallado

Cambio en una frase

PersistenciaService deja de conocer cómo se guardan físicamente los datos, cómo se construyen los validadores interceptados y dónde se almacena el mensaje de validación. Solo solicita esas capacidades mediante interfaces.

En el AS-IS, PersistenciaService conocía directamente el entorno web, las rutas y operaciones físicas de archivos, además de las implementaciones concretas de validadores interceptados y del resultado de validación HTTP. En el TO-BE esas necesidades se expresan mediante IAlmacenamientoHacienda, IValidadoresGuardado e IResultadoValidacion, de modo que el servicio depende de capacidades y no de tecnologías concretas.

IAlmacenamientoHacienda — separar qué se guarda de cómo se guarda

IAlmacenamientoHacienda define las tres capacidades físicas que PersistenciaService necesita sin comprometerse con una tecnología concreta: guardar un conjunto de registros, recuperar los registros asociados a un nombre lógico y comprobar si esos datos existen. El contrato describe necesidades del sistema y no menciona archivos, rutas ni directorios.

PersistenciaService sigue sabiendo qué información está procesando y conserva la preparación de las líneas actuales. Lo que deja de saber es cómo esas líneas llegan al almacenamiento físico.

No se denomina IArchivos porque eso convertiría la tecnología actual en parte del contrato.

IAlmacenamientoHacienda expresa la necesidad del sistema sin asumir que siempre será un archivo plano.

AlmacenamientoArchivosHacienda — implementación actual

AlmacenamientoArchivosHacienda es la implementación actual de IAlmacenamientoHacienda. Esta clase concentra el conocimiento del directorio físico, del entorno web y de las operaciones de guardar, cargar y comprobar existencia. PersistenciaService solicita esas capacidades sin conocer que, en la implementación actual, se resuelven mediante archivos planos.

Esta clase recibe exactamente el conocimiento físico que sale de PersistenciaService:

IWebHostEnvironment, cálculo de ruta, directorio Datos y llamadas a File, Directory y Path. También conserva los mismos archivos .txt y la misma semántica de lectura/escritura.

Ejemplo conceptual: PersistenciaService solicita guardar "Ventas"; AlmacenamientoArchivosHacienda es quien sabe que la implementación actual corresponde al mismo Datos/Ventas.txt. El archivo resultante no cambia.

IValidadoresGuardado — PersistenciaService no conoce el proveedor concreto

IValidadoresGuardado agrupa las cuatro capacidades de validación que PersistenciaService necesita para Potrero, Res, Vacuna y Venta, cada una expresada mediante IValidador<T>.

ValidadoresPersistenciaInterceptados proporciona esas capacidades y mantiene internamente el detalle de Castle, interceptores y proxies, sin exponerlo al consumidor.

H-13 ya había separado la creación lazy de Castle y los proxies en ValidadoresPersistenciaInterceptados. H-15 completa DIP: PersistenciaService deja de depender de esa clase concreta y recibe IValidadoresGuardado.

Esto no reintroduce el problema ISP de H-12. Ningún ValidadorRes, ValidadorPotrero, ValidadorVacuna o ValidadorVenta está obligado a validar otros tipos; cada propiedad conserva el contrato estrecho IValidador<T>. El consumidor PersistenciaService sí necesita las cuatro capacidades porque guarda esos cuatro dominios.

IResultadoValidacion — ocultar HttpContext al módulo de alto nivel

IResultadoValidacion define únicamente la capacidad de obtener el texto final de una validación cuando se dispone de un mensaje alternativo. ResultadoValidacionHttpContext implementa ese contrato utilizando el contexto HTTP actual. PersistenciaService depende de la abstracción y desconoce el lugar técnico donde se almacena el resultado.

PersistenciaService solicita únicamente el resultado de validación. ResultadoValidacionHttpContext sigue leyendo HttpContext.Items["ResultadoValidacion"] y conserva exactamente el mismo fallback, pero ese detalle queda oculto detrás de IResultadoValidacion.

IResultadoValidacion no tiene relación con los notificadores de peso, potrero o incorporación de reses. Aquellos comunican eventos del negocio; esta interfaz solo entrega el texto producido por el flujo de validación previo al guardado.

Qué dependencias conserva PersistenciaService

Después de H-15, PersistenciaService expresa sus colaboraciones mediante abstracciones. Utiliza IAlmacenamientoHacienda para el medio físico, IValidadoresGuardado para obtener validadores, IResultadoValidacion para el mensaje de validación y reutiliza IValidador<Usuario>, ICargaVacuna e IReglasTipoPotrero para capacidades ya abstraídas en hallazgos anteriores.

No se crean interfaces duplicadas para capacidades que ya estaban correctamente abstraídas. H-15 introduce únicamente las tres abstracciones que faltaban para separar a PersistenciaService de infraestructura concreta.

Qué ocurre con los diferentes tipos de guardado

Potreros, Reses, Ventas, Vacunas, VacunasAplicadas y Usuarios no generan seis interfaces de almacenamiento. PersistenciaService conserva la responsabilidad de preparar la representación actual de cada dominio; IAlmacenamientoHacienda recibe registros ya preparados y decide cómo guardarlos físicamente.

La responsabilidad se distribuye por capas: las entidades del dominio llegan a PersistenciaService, que prepara exactamente la misma representación lógica; luego solicita a IAlmacenamientoHacienda que la conserve; y la implementación actual AlmacenamientoArchivosHacienda decide que esa operación se materializa en los mismos archivos de texto existentes. Así se separa qué se guarda de cómo se guarda.

Tampoco se inventan operaciones como Agregar, Actualizar o Sobrescribir si el sistema actual no las diferencia. La interfaz conserva la semántica observable existente; una nueva capacidad solo se agregaría cuando aparezca una necesidad real del negocio.

Matriz explícita de inversión de dependencias

Campo exigido	H-15
Módulo de alto nivel	PersistenciaService: coordina guardar/cargar el estado y necesita capacidades de almacenamiento y validación.
Módulo de bajo nivel / detalle concreto	AlmacenamientoArchivosHacienda; ValidadoresPersistenciaInterceptados; ResultadoValidacionHttpContext.
Abstracción que desacopla	IAmacenamientoHacienda; IValidadoresGuardado; IResultadoValidacion. También se reutilizan IValidador<Usuario>, ICargaVacuna e IReglasTipoPotrero.
Composition root	p_mvcHacienda/Program.cs, método Main: registra IAmacenamientoHacienda→AlmacenamientoArchivosHacienda, IValidadoresGuardado→ValidadoresPersistenciaInterceptados e IResultadoValidacion→ResultadoValidacionHttpContext; el contenedor construye PersistenciaService.

Por qué Program sí puede conocer los concretos

Program es el composition root. Su responsabilidad es decidir qué implementación concreta satisface cada interfaz al arrancar la aplicación. Que Program conozca AlmacenamientoArchivosHacienda, ValidadoresPersistenciaInterceptados y ResultadoValidacionHttpContext no viola DIP; evita que ese conocimiento se propague al módulo de alto nivel.

Program actúa como composition root y construye las implementaciones concretas de almacenamiento por archivos, validadores interceptados y resultado de validación basado en HttpContext. Luego las suministra a PersistenciaService mediante IAmacenamientoHacienda, IValidadoresGuardado e IResultadoValidacion. Así, el conocimiento de los concretos queda concentrado en el arranque.

Preservación estricta del comportamiento

- Se conservan los mismos archivos .txt, nombres y directorio Datos.
- Se conservan los mismos delimitadores, orden de campos y valores serializados.
- GuardarX y CargarX conservan sus firmas públicas y el mismo orden funcional.
- Las validaciones siguen utilizando los mismos validadores y proxies.
- La inicialización de interceptor/proxies continúa siendo lazy dentro de ValidadoresPersistenciaInterceptados.
- HttpContext.Items["ResultadoValidacion"] sigue siendo leído por ResultadoValidacionHttpContext con los mismos fallbacks.
- Los mensajes, excepciones y resultados actuales no cambian.
- H-15 no cambia la lógica OCP de carga por tipos resuelta en H-14.

Qué se gana e impacto para negocio y mantenimiento

Cambio futuro	Antes	Después de H-15	Impacto
---------------	-------	-----------------	---------

Cambio futuro	Antes	Después de H-15	Impacto
Cambiar mecanismo físico de almacenamiento	modificar PersistenciaService y volver a probar una clase que también coordina validación	crear/sustituir una implementación de IAlmacenamientoHacienda	menor tiempo de cambio y menor riesgo sobre la lógica central.
Cambiar Castle/proxies	PersistenciaService quedaba atado al proveedor concreto	cambia ValidadoresPersistenciaInterceptados detrás de IValidadoresGuardado	pruebas más focalizadas y menor propagación del cambio.
Cambiar dónde se obtiene el mensaje de validación	afectar al consumidor de alto nivel	sustituir IResultadoValidacion	menor acoplamiento a la tecnología web.
Probar PersistenciaService	requerir infraestructura física/concreta	usar implementaciones controladas de sus interfaces	menor costo y tiempo de pruebas.

H-16 — PotreroService mezcla responsabilidades y depende de concretos

Problemas señalados en el hallazgo

PotreroService mezcla creación y agregado de reses con consultas, estadísticas, localización de reglas y persistencia, y además depende de Hacienda y PersistenciaService concretos.

Solución aprobada

Cambio detallado

División SRP final: PotreroService se separa en cuatro responsabilidades

Elemento	Responsabilidad final	Qué sale de PotreroService
PotreroService	Coordinar operaciones que cambian el estado: CrearPotrero y AgregarRes.	Conserva únicamente las dos operaciones de cambio y sus reglas/orden observable.
ConsultaPotreros	Listar y buscar potreros.	ObtenerTodosLosPotreros y ObtenerPotreroPorIdentificacion.
EstadisticasPotreros	Calcular las mismas estadísticas actuales.	ObtenerEstadisticas.
TiposPotreroDisponibles	Localizar IReglasTipoPotrero por el mismo TipoPotrero textual.	El diccionario/mapa de reglas y su lookup.

No se divide por cantidad de métodos sino por razones de cambio. Una modificación en una estadística no debe obligar a revisar la creación de potreros; cambiar el modo de localizar una regla no debe tocar la lógica de búsqueda; y cambiar almacenamiento no debe obligar a modificar el servicio de aplicación.

IEstadoPotreros — quién lo implementa y quién lo usa

IEstadoPotreros expone una vista de solo lectura de los potreros existentes. Hacienda implementa este contrato porque continúa siendo la propietaria de la colección; los consumidores pueden consultar el estado sin recibir toda la clase Hacienda ni obtener contractualmente una colección mutable.

Solo Hacienda realiza IEstadoPotreros porque es la clase que posee l_potreros/L_potreros. ConsultaPotreros no la implementa: únicamente la recibe y la utiliza. EstadisticasPotreros ni siquiera necesita conocer esta interfaz, porque obtiene los potreros mediante IConsultaPotreros.

La vista IReadOnlyCollection evita que una consulta obtenga una colección mutable como contrato. Hacienda conserva internamente su estado original; no cambia la asociación Hacienda–Potrero ni su multiplicidad 0..*.

IConsultaPotreros — comportamiento real de consulta

IConsultaPotreros reúne las operaciones de consulta necesarias para obtener todos los potreros o localizar uno por su identificación. ConsultaPotreros implementa este contrato y conserva los mismos criterios y resultados de búsqueda que antes estaban dentro de PotreroService.

Aquí sí existe un comportamiento claramente nombrado: listar y buscar. La lógica se mueve desde PotreroService a ConsultaPotreros manteniendo la misma comparación por identificación y el mismo resultado nulo cuando no existe.

EstadisticasPotreros depende de IConsultaPotreros y solicita los potreros que necesita para calcular. No accede directamente a Hacienda ni a su colección interna.

IRegistroPotrero — modificación mínima del estado sin devolver H-01 a Hacienda

IRegistroPotrero define únicamente la capacidad de incorporar un Potrero al estado existente. Hacienda implementa esta interfaz porque es la propietaria de la colección. PotreroService puede registrar el objeto ya construido sin recuperar por ello las demás capacidades de Hacienda.

PotreroService sigue creando el Potrero y coordinando el caso de uso. Hacienda únicamente recibe el objeto ya construido y lo incorpora a su estado. Esto no reabre H-01: Hacienda no recupera creación, reglas de tipo, validación, mensajes, notificaciones ni persistencia; solo conserva la propiedad del agregado.

ITiposPotreroDisponibles — sacar la localización de reglas de PotreroService

ITiposPotreroDisponibles representa la capacidad de obtener las reglas correspondientes a un tipo textual de potrero. TiposPotreroDisponibles implementa esa interfaz y conserva internamente el diccionario que relaciona cada texto con una IReglasTipoPotrero. PotreroService utiliza el contrato sin conocer la estructura concreta de ese registro.

Las reglas ReglaTipoPotreroTernero, ReglaTipoPotreroCebon y ReglaTipoPotreroNovillo permanecen en el grupo de reglas aprobado en H-04/H-05. TiposPotreroDisponibles NO es otra regla: únicamente localiza la regla correspondiente al texto recibido.

PotreroService deja de consultar directamente un diccionario interno de reglas. Después del cambio solicita a ITiposPotreroDisponibles la regla asociada al texto recibido. La selección continúa produciendo la misma IReglasTipoPotrero, pero el mecanismo de localización queda fuera del servicio y puede cambiar sin modificar su lógica de creación o incorporación.

El mismo mapa TipoPotrero → IReglasTipoPotrero se configura en Program y se entrega a TiposPotreroDisponibles. No se introduce switch, enum ni cadena if/else equivalente.

IActualizacionPotreros e IActualizacionReses — sustituir PersistenciaService concreto

IActualizacionPotreros e IActualizacionReses expresan por separado las capacidades necesarias para reflejar en persistencia el estado actualizado de potreros y reses. PersistenciaService implementa ambos contratos, permitiendo que los casos de uso soliciten únicamente la actualización que realmente necesitan sin depender de toda la clase de persistencia.

PotreroService no necesita todo PersistenciaService. Solo necesita las capacidades de actualización que utiliza en sus flujos actuales. Por eso depende de IActualizacionPotreros para reflejar cambios de potreros y de IActualizacionReses para reflejar cambios de reses, conservando el mismo punto lógico de persistencia, retornos, mensajes y orden observable.

H-15 sigue vigente debajo de esta frontera: PersistenciaService, a su vez, utiliza IAlmacenamientoHacienda y no expone al PotreroService los archivos físicos.

PotreroService final

PotreroService queda concentrado en los dos casos de uso que modifican el estado: crear un potrero y agregar una res. Para realizarlos consume IConsultaPotreros, IRegistroPotrero, ITiposPotreroDisponibles, IActualizacionPotreros, IActualizacionReses e INotificacionIncorporacionRes. Cada colaboración representa una necesidad específica y evita que el servicio vuelva a conocer componentes concretos amplios.

CrearPotrero solicita la regla mediante ITiposPotreroDisponibles, construye el mismo Potrero, lo incorpora mediante IRegistroPotrero y conserva la misma actualización mediante IActualizacionPotreros. AgregarRes localiza el Potrero mediante IConsultaPotreros, conserva la incorporación y notificación aprobadas en H-06 y refleja las reses mediante IActualizacionReses.

Adaptación de consumidores sin anticipar H-30/H-32

Al mover físicamente los métodos de consulta/estadísticas, los consumidores que los invocaban deben recibir las nuevas fronteras para que el sistema siga compilando y se conserve el mismo flujo web. PotreroController puede usar IConsultaPotreros y EstadisticasPotreros para Index/Details/estadísticas; ResController y VacunaController pueden usar IConsultaPotreros donde antes consultaban a través de PotreroService.

H-16 no cerró por sí solo H-30 ni H-32.

Matriz explícita de inversión de dependencias

Campo exigido	H-16
Módulo de alto nivel	PotreroService para creación/agregado; ConsultaPotreros para consulta; EstadisticasPotreros para cálculo estadístico.
Módulo de bajo nivel / detalle concreto	Hacienda como proveedor concreto del estado/registro; PersistenciaService como guardado concreto; TiposPotreroDisponibles como implementación de localización de reglas.
Abstracción que desacopla	IEstadoPotreros; IRegistroPotrero; IConsultaPotreros; ITiposPotreroDisponibles; IActualizacionPotreros; IActualizacionReses. Se reutilizan IReglasTipoPotrero e INotificacionIncorporacionRes.
Composition root	p_mvcHacienda/Program.cs, método Main: comparte Hacienda como singleton detrás de IEstadoPotreros/IRegistroPotrero, registra IConsultaPotreros, ITiposPotreroDisponibles y PersistenciaService detrás de IActualizacionPotreros/IActualizacionReses; el contenedor resuelve PotreroService y EstadisticasPotreros.

Preservación estricta del comportamiento

- CrearPotrero y AgregarRes conservan los mismos parámetros, reglas, efectos y resultados observables.

- ObtenerTodosLosPotreros, ObtenerPotreroPorIdentificacion y ObtenerEstadisticas mantienen su lógica; solo cambian de clase responsable.
- Se conserva la misma comparación/normalización de identificaciones que use el flujo actual.
- Las reglas de tipo, edad, capacidad y construcción aprobadas en H-04/H-05/H-09–H-11 no cambian.
- La coordinación de notificaciones H-06 y su orden no cambian.
- Las mismas actualizaciones se ejecutan en el mismo punto lógico; solo se invocan mediante IActualizacionPotreros e IActualizacionReses.
- No se introducen switch, enums de variación ni cadenas if/else equivalentes.
- No se elimina ni cambia ningún mensaje o excepción observable.

Qué se gana e impacto para negocio y mantenimiento

Cambio futuro	Antes	Después de H-16	Impacto
Cambiar una estadística	tocar PotreroService que también crea/agrega/persiste	modificar EstadisticasPotreros	menor riesgo de afectar operaciones de negocio y menos pruebas de regresión.
Cambiar búsqueda/consulta	tocar la misma clase de creación	modificar ConsultaPotreros	menor tiempo y costo de mantenimiento.
Cambiar localización de tipos	modificar PotreroService	modificar TiposPotreroDisponibles o la composición	menor radio de cambio y PotreroService más estable.
Cambiar persistencia	PotreroService dependía de PersistenciaService concreto	mantener IActualizacionPotreros/IActualizacionReses y sustituir o adaptar la implementación	menor acoplamiento y pruebas más aisladas.
Cambiar propietario/forma de exponer estado	PotreroService conocía Hacienda	consumir IConsultaPotreros/IRegistroPotrero	menor propagación del cambio de dominio concreto.

H-17 — ResService depende directamente de Hacienda y PersistenciaService

Problemas señalados en el hallazgo

ResService depende directamente de Hacienda para acceder a los potreros y recibe PersistenciaService aunque esa dependencia no participa en su comportamiento.

Solución aprobada

Cambio detallado

Cambio aprobado

ResService elimina los campos y parámetros concretos Hacienda y PersistenciaService. En su lugar recibe únicamente IConsultaPotreros, porque esa es la capacidad que realmente necesita para obtener los potreros y, desde ellos, acceder a las reses.

PersistenciaService se elimina sin interfaz sustituta. Como no se utiliza en ResService, crear una abstracción para conservar una dependencia innecesaria sería artificial y mantendría acoplamiento sin razón de negocio.

Responsabilidad y relación entre las piezas

Hacienda conserva el estado de la Hacienda. ConsultaPotreros sabe buscar y entregar los potreros y realiza IConsultaPotreros. ResService trabaja con las reses y consume IConsultaPotreros para llegar a ellas. No comparten una responsabilidad ni existe herencia entre ConsultaPotreros y ResService: una clase proporciona una capacidad y la otra la utiliza.

Flujo TO-BE: Hacienda → estado de potreros; ConsultaPotreros → IConsultaPotreros; ResService → consume IConsultaPotreros → obtiene las reses contenidas en los potreros.

Matriz explícita de inversión de dependencias

Campo exigido	H-17
Módulo de alto nivel	ResService.
Módulo de bajo nivel / detalle concreto	Hacienda era la dependencia concreta usada para acceder a potreros. PersistenciaService era concreto e inyectado, pero no utilizado.
Abstracción que desacopla	IConsultaPotreros, reutilizada de H-16.
Composition root	p_mvcHacienda/Program.cs, método Main: registra ConsultaPotreros como IConsultaPotreros y el contenedor suministra esa abstracción a ResService. En el código final, H-32 añade además IActualizacionReses sin reintroducir Hacienda ni PersistenciaService concretos.

Preservación estricta del comportamiento

No cambian los resultados de ObtenerTodasLasReses(), BuscarRes(...), AlimentarRes(...) ni ObtenerEstadisticas(); tampoco cambian reglas, mensajes, excepciones ni orden de ejecución. Solo cambia la vía interna utilizada para obtener los potreros.

Las comprobaciones explícitas Ternero/Cebon/Novillo que existen dentro de ObtenerEstadisticas() se conservan deliberadamente en H-17. Esas decisiones pertenecen a H-18 (OCP) y se tratarán por separado para no mezclar hallazgos.

Qué se gana e impacto para negocio y mantenimiento

Tiempo: cambios internos en Hacienda dejan de obligar automáticamente a modificar ResService mientras se mantenga el contrato de consulta. Costo: se elimina una dependencia de PersistenciaService que no aporta comportamiento, reduciendo mantenimiento y configuración. Riesgo: las pruebas de ResService pueden concentrarse en su necesidad real —consultar potreros para operar sobre reses— con menor superficie de regresión.

H-18 — ResService calcula estadísticas mediante tipos concretos

Problemas señalados en el hallazgo

ResService calcula estadísticas preguntando explícitamente si cada res es Ternero, Cebon o Novillo, por lo que una nueva categoría obliga a modificar el método.

Solución aprobada

Cambio detallado

Cambio aprobado: TipoRes

Se agrega en la clase abstracta Res una propiedad pública, de solo lectura y polimórfica llamada TipoRes. No es un enum, no es un código de selección y no se utiliza para decidir qué objeto crear. Expresa una característica de negocio que toda Res puede informar sobre sí misma.

Elemento	TipoRes TO-BE	Responsabilidad
Res	Propiedad abstracta TipoRes de solo lectura, común a toda la jerarquía.	Define que toda Res debe informar su categoría sin revelar su clase concreta al consumidor.
Ternero	"Terneros"	Entrega la misma etiqueta observable que hoy utiliza el diccionario de estadísticas.
Cebon	"Cebones"	Entrega la misma etiqueta observable que hoy utiliza el diccionario de estadísticas.
Novillo	"Novillos"	Entrega la misma etiqueta observable que hoy utiliza el diccionario de estadísticas.

Los valores “Terneros”, “Cebones” y “Novillos” se conservan deliberadamente porque son exactamente las claves actuales expuestas por ObtenerEstadisticas(). TipoRes no reemplaza IReglasTipoPotrero. TipoPotrero: aquella propiedad sigue atendiendo la localización/creación de potreros; TipoRes solo informa la categoría estadística de una Res ya existente.

Cómo funciona ObtenerEstadisticas() después del cambio

ResService obtiene las reses exactamente por la misma vía de H-17. Luego prepara el mismo diccionario observable con TotalReses, Terneros, Cebones y Novillos; los tres contadores se inicializan en cero para que sigan apareciendo aun cuando no exista ninguna res de una categoría. Después agrupa las reses por Res.TipoRes y actualiza el conteo de cada grupo de forma genérica. Finalmente agrega PesoPromedio usando exactamente la misma fórmula actual.

Clave observable	AS-IS	TO-BE H-18
TotalReses	todasLasReses.Count	Se conserva exactamente.
Terneros	Conteo mediante comprobación directa de la clase Ternero.	Conteo del grupo cuya Res.TipoRes es “Terneros”.
Cebones	Conteo mediante comprobación directa de la clase Cebon.	Conteo del grupo cuya Res.TipoRes es “Cebones”.
Novillos	Conteo mediante comprobación directa de la clase Novillo.	Conteo del grupo cuya Res.TipoRes es “Novillos”.

Clave observable	AS-IS	TO-BE H-18
PesoPromedio	Average(Peso) si hay reses; 0 en caso contrario	Se conserva exactamente la misma condición y cálculo.

Con los tipos actuales, el orden observable se mantiene: TotalReses → Terneros → Cebones → Novillos → PesoPromedio. El cambio afecta únicamente la forma interna de obtener los tres conteos.

Por qué ahora cumple OCP

Si en el futuro aparece un nuevo subtipo de Res, ese subtipo debe sobrescribir TipoRes con su propia categoría. ObtenerEstadisticas() no necesita agregar “res is NuevoTipo”, un switch, un enum ni una cadena if/else. La agrupación genérica incorpora esa categoría sin modificar las comprobaciones existentes porque ya no existen comprobaciones por clase concreta.

La diferencia con una simple sustitución por cadenas es importante: ResService no compara TipoRes contra “Ternero”, “Cebon” o “Novillo” para seleccionar comportamiento. Solo agrupa por el valor que cada objeto proporciona polimórficamente. Por ello no se traslada el mismo problema de OCP desde “is” hacia comparaciones de texto.

Por qué no se crea una interfaz nueva

No se introduce una interfaz adicional porque TipoRes es una característica intrínseca de la jerarquía Res y las clases Ternero, Cebon y Novillo ya son especializaciones válidas de Res. A diferencia del error de interfaces revisado en H-16, aquí sí existe un contrato común real que todos los subtipos pueden cumplir: toda Res puede informar su tipo. ResService consume el contrato de Res; no hereda ni realiza una interfaz nueva.

Verificación de la herencia conservada	Resultado H-18
Por qué herencia	Ternero, Cebon y Novillo siguen siendo Res; TipoRes describe una propiedad común de esa misma entidad.
Por qué no composición	Crear un objeto auxiliar solo para devolver el nombre del tipo añadiría delegación sin aportar una responsabilidad independiente.
Precondiciones	Leer TipoRes no requiere una precondición adicional.
Postcondiciones	Cada subtipo devuelve siempre su categoría actual: Terneros, Cebones o Novillos.
Invariantes	TipoRes permanece coherente con el subtipo durante toda la vida de la Res.
Excepciones	Los getters de TipoRes no introducen excepciones nuevas.
Sustituibilidad	Cualquier Ternero/Cebon/Novillo tratado como Res responde al mismo miembro TipoRes sin comprobación de clase concreta.

Preservación estricta del comportamiento

No cambian las firmas públicas existentes de ResService, los nombres de las claves del diccionario, el orden de las cinco estadísticas actuales, el valor de TotalReses, los conteos de Terneros/Cebones/Novillos, el cálculo de PesoPromedio, el valor 0 cuando no existen reses, los mensajes, las excepciones ni el orden de los casos de uso. TipoRes agrega información polimórfica a la jerarquía, pero no modifica edad, peso, vacunas, alimentación ni construcción.

Qué se gana e impacto para negocio y mantenimiento

Dimensión	Beneficio H-18
-----------	----------------

Dimensión	Beneficio H-18
Tiempo	Agregar una categoría futura de Res deja de exigir editar <code>ObtenerEstadisticas()</code> ; el cambio se concentra en el nuevo subtipo.
Costo	Disminuye el número de clases existentes que deben modificarse y volver a probarse ante una extensión de la jerarquía.
Riesgo	Se evita reabrir las condiciones de conteo de Terneros, Cebones y Novillos cada vez que aparece una nueva categoría, reduciendo regresiones en estadísticas ya estables.
Flexibilidad	La estadística trabaja con el contrato de Res y puede incorporar nuevas categorías sin conocer sus clases concretas.

H-19 — UsuarioService depende directamente de PersistenciaService

Problemas señalados en el hallazgo

UsuarioService depende directamente de PersistenciaService para cargar y guardar usuarios, acoplando el servicio a una implementación concreta más amplia de lo necesario.

Solución aprobada

Cambio detallado

Cambio aprobado

Se crean dos capacidades estrechas: `ICargaUsuarios`, con `CargarUsuarios()`, e `IGuardadoUsuarios`, con `GuardarUsuarios(...)`. `PersistenciaService` conserva exactamente esas operaciones y realiza ambas interfaces. `UsuarioService` elimina su dependencia directa a `PersistenciaService` y recibe únicamente las dos capacidades que utiliza.

Se usan dos interfaces y no una `IPersistenciaUsuarios` porque cargar y guardar son necesidades distintas. Esta separación permite que, cuando H-20 divida `UsuarioService` por responsabilidades, cada nueva pieza reciba únicamente la capacidad que realmente necesite, evitando volver a crear una interfaz amplia.

Responsabilidad y relación entre las piezas

`PersistenciaService` proporciona la capacidad actual de cargar y guardar usuarios. `ICargaUsuarios` e `IGuardadoUsuarios` expresan esas capacidades. `UsuarioService` las consume; no las implementa y no hereda comportamiento de `PersistenciaService`. No existe una responsabilidad común entre ambas clases: una proporciona persistencia y la otra coordina actualmente operaciones de usuarios.

Flujo TO-BE: `UsuarioService` → `ICargaUsuarios` / `IGuardadoUsuarios` ← `PersistenciaService`. Program conoce la implementación concreta y conecta las abstracciones en el composition root.

Contratos incorporados

`ICargaUsuarios` representa únicamente la capacidad de obtener los usuarios almacenados, mientras que `IGuardadoUsuarios` representa la capacidad de persistir el conjunto actualizado. `PersistenciaService` proporciona ambas, pero los consumidores pueden depender de carga o guardado por separado según su responsabilidad.

Las firmas reproducen las operaciones existentes del sistema. No se modifica el tipo de colección, el valor retornado ni la semántica de carga/guardado dentro de este hallazgo.

Matriz explícita de inversión de dependencias

Campo exigido	H-19
Módulo de alto nivel	UsuarioService.
Módulo de bajo nivel / detalle concreto	PersistenciaService concreto.
Abstracción que desacopla	ICargaUsuarios e IGuardadoUsuarios.
Composition root	p_mvcHacienda/Program.cs, método Main: registra la misma instancia de PersistenciaService como ICargaUsuarios e IGuardadoUsuarios. H-20 elimina posteriormente UsuarioService, pero conserva estas capacidades para el módulo de usuarios.

Preservación estricta del comportamiento

- CargarUsuarios() se invoca en el mismo punto lógico y conserva su resultado actual.
- GuardarUsuarios(...) se invoca en los mismos flujos actuales y conserva retorno, mensajes y excepciones.
- No cambian archivos, nombres, formato, delimitadores, orden de datos ni mecanismo físico de almacenamiento.
- No cambian la lista _usuarios, autenticación, creación, consultas, estadísticas ni generación de Claims.
- No se introduce switch, enum, if/else de selección ni patrón de diseño.
- H-20 se resuelve posteriormente separando las responsabilidades internas de UsuarioService; H-19 conserva como resultado las abstracciones ICargaUsuarios e IGuardadoUsuarios.

Qué se gana e impacto para negocio y mantenimiento

Tiempo: cambios en la implementación concreta de persistencia dejan de propagarse automáticamente a UsuarioService mientras se mantengan los dos contratos. Costo: las pruebas pueden sustituir carga y guardado de usuarios sin configurar toda PersistenciaService. Riesgo: se reduce la superficie de regresión al limitar el acoplamiento a dos capacidades explícitas.

H-20 — UsuarioService concentra gestión, autenticación, consultas, estadísticas y Claims

Problemas señalados en el hallazgo

UsuarioService concentra carga y estado de usuarios, creación, autenticación, consultas, estadísticas y generación de Claims en una sola clase.

Solución aprobada

Cambio detallado

Cambio aprobado: UsuarioService desaparece como clase concentradora

UsuarioService se elimina del TO-BE. Sus operaciones no se pierden: cada operación se traslada inmediatamente a la clase cuya única razón de cambio coincide con esa conducta. La colección que antes estaba en `_usuarios` se mueve a `UsuariosRegistradosHacienda` y se comparte como una única instancia creada en `Program`.

No se crea una clase `CargaUsuarios` adicional. `Program` ejecuta la misma carga inicial mediante `ICargaUsuarios`, construye `UsuariosRegistradosHacienda` con esa lista y comparte ese mismo objeto con las cinco responsabilidades que necesitan trabajar sobre los usuarios actuales. De esta forma se conserva el efecto que tenía la lista estática compartida sin mantener una clase multifunción.

División SRP final: seis fronteras

Elemento	Responsabilidad única	Operaciones/estado que recibe
<code>UsuariosRegistradosHacienda</code>	Mantener la única colección compartida de usuarios cargados.	<code>_usuarios / Usuarios</code>
<code>CreacionUsuarioHacienda</code>	Validar la solicitud de creación, crear el usuario, incorporarlo y solicitar el mismo guardado.	<code>CrearUsuario(...)</code>
<code>ConsultaUsuariosHacienda</code>	Listar y buscar usuarios sin modificar estado.	<code>ObtenerTodosLosUsuarios()</code> , <code>BuscarUsuario(...)</code>
<code>AutenticacionUsuarioHacienda</code>	Comprobar credenciales con las reglas actuales de esa operación.	<code>AutenticarUsuario(...)</code>
<code>EstadisticasUsuariosHacienda</code>	Calcular las mismas estadísticas actuales.	<code>ObtenerEstadisticas()</code>
<code>InicioSesionUsuario</code>	Conservar <code>ValidateUserAsync</code> y la integración web que genera los mismos Claims.	<code>ValidateUserAsync(...)</code>

Por qué la validación de creación permanece con `CreacionUsuarioHacienda`

La validación señalada en el hallazgo no se separa en una clase adicional porque, en el código observado, protege exclusivamente el caso de uso `CrearUsuario`. Es una precondition propia de esa responsabilidad. Separarla sin una segunda necesidad independiente produciría una frontera artificial y aumentaría clases sin reducir una razón de cambio real. El cuerpo de `CrearUsuario`, incluidas sus validaciones, mensajes, excepciones, orden y llamada de guardado, se traslada completo.

Estado compartido: qué comparten realmente las nuevas clases

Las seis clases no heredan entre sí y no implementan una interfaz común. Lo único compartido es la necesidad de trabajar sobre la misma colección de usuarios que antes estaba en `UsuarioService`. `UsuariosRegistradosHacienda` es el único dueño de esa colección; `CreacionUsuarioHacienda`, `ConsultaUsuariosHacienda`, `AutenticacionUsuarioHacienda`, `EstadisticasUsuariosHacienda` e `InicioSesionUsuario` la consumen.

Esto corrige el riesgo de repetir el error de interfaces detectado en H-16: compartir datos no significa compartir comportamiento. Por eso H-20 no crea `IUsuarios`, `IUsuarioService` ni una interfaz amplia. La relación es de composición/uso sobre un estado común, no de herencia.

Mapeo exacto de las operaciones

Antes en UsuarioService	TO-BE
CargarUsuarios()	Program usa ICargaUsuarios al inicio y entrega la misma lista a UsuariosRegistradosHacienda.
CrearUsuario(...)	CreacionUsuarioHacienda.CrearUsuario(...)
AutenticarUsuario(...)	AutenticacionUsuarioHacienda.AutenticarUsuario(...)
ObtenerTodosLosUsuarios()	ConsultaUsuariosHacienda.ObtenerTodosLosUsuarios()
BuscarUsuario(...)	ConsultaUsuariosHacienda.BuscarUsuario(...)
ObtenerEstadisticas()	EstadisticasUsuariosHacienda.ObtenerEstadisticas()
ValidateUserAsync(...)	InicioSesionUsuario.ValidateUserAsync(...)

Preservación estricta del comportamiento

- La carga inicial sigue utilizando ICargaUsuarios y ocurre una vez en el mismo arranque de la aplicación; no se introduce una segunda colección de usuarios.
- La misma instancia de usuarios se comparte entre creación, consultas, autenticación, estadísticas e inicio de sesión, preservando el efecto del estado compartido anterior.
- CrearUsuario conserva exactamente sus validaciones, orden, creación del objeto, incorporación a la colección, retorno, mensajes, excepciones y solicitud de guardado mediante IGuardadoUsuarios.
- AutenticarUsuario conserva sus propias comparaciones actuales. No se fuerza a ValidateUserAsync a reutilizar este método porque no se asume que ambas operaciones tengan idénticas reglas de comparación sin evidencia del código.
- ObtenerTodosLosUsuarios, BuscarUsuario y ObtenerEstadisticas conservan firmas, resultados y orden observable.
- ValidateUserAsync conserva el mismo resultado asíncrono y la misma generación de Claims; solo cambia la clase responsable.
- No se cambia el formato de persistencia, usuarios almacenados, contraseñas, roles, Claims, mensajes ni excepciones.
- No se introduce switch, enum, cadena if/else de selección ni patrón de diseño.

Adaptación de los consumidores

AccountController deja de recibir UsuarioService y recibe InicioSesionUsuario, porque su flujo de Login necesita ValidateUserAsync. UsuarioController deja de recibir la clase multifunción y recibe ConsultaUsuariosHacienda para listado/búsqueda y CreacionUsuarioHacienda para el POST de creación. Las acciones web, rutas y resultados no cambian; solo cambia a qué responsabilidad delegan internamente.

AutenticacionUsuarioHacienda y EstadisticasUsuariosHacienda quedan disponibles como responsabilidades independientes para los consumidores que ya utilizan esas operaciones. Program construye y registra todas las piezas usando la misma instancia de UsuariosRegistradosHacienda.

Relación con H-19 y composition root

PersistenciaService permanece. Sigue realizando ICargaUsuarios e IGuardadoUsuarios. Después de H-20, Program utiliza ICargaUsuarios para obtener la lista inicial y CreacionUsuarioHacienda utiliza IGuardadoUsuarios después de crear un usuario. Por tanto H-19 no se deshace: la clase concreta de persistencia continúa detrás de las abstracciones.

Program es el composition root: obtiene los usuarios mediante ICargaUsuarios, crea un único UsuariosRegistradosHacienda y entrega esa misma instancia a CreacionUsuarioHacienda,

ConsultaUsuariosHacienda, AutenticacionUsuarioHacienda, EstadisticasUsuariosHacienda e InicioSesionUsuario. También conecta IGuardadoUsuarios con CreacionUsuarioHacienda.

Herencia y composición

H-20 no agrega ninguna relación de herencia. No existe una relación “es-un” entre creación, consulta, autenticación, estadísticas e inicio de sesión. Se utiliza composición/uso porque todas trabajan sobre el mismo estado de usuarios pero tienen responsabilidades distintas. Por tanto no aparecen nuevas precondiciones de sustitución LSP ni una jerarquía artificial.

Las clases originales no relacionadas con el cambio permanecen negras.

Qué se gana e impacto para negocio y mantenimiento

Tiempo: modificar Claims o integración de inicio de sesión ya no obliga a revisar creación, estadísticas y consultas. Costo: las pruebas se concentran en una responsabilidad más pequeña y el guardado solo se configura donde realmente se necesita. Riesgo: un cambio en reglas de creación no afecta el código de autenticación o estadísticas y un cambio de reporte no toca seguridad. Esto reduce la superficie de regresión y el costo de evolución del módulo de usuarios.

Aclaración de H-20 — por qué existe UsuariosRegistradosHacienda

Sí es necesaria una única frontera que sea dueña del estado compartido después de dividir UsuarioService, salvo que se inyectara una List<Usuario> cruda en todas las clases. Se conserva UsuariosRegistradosHacienda porque da un dueño explícito a la misma colección que antes compartía UsuarioService y evita crear copias o estados divergentes.

Contenido exacto: solo mantiene _usuarios : List<Usuario>, expone Usuarios : List<Usuario> como la misma instancia y recibe esa lista en su constructor. No autentica, no crea usuarios, no consulta, no calcula estadísticas, no genera Claims y no persiste. Es estado compartido, no un nuevo servicio de negocio.

Esta frontera preserva la conducta: CreacionUsuarioHacienda, ConsultaUsuariosHacienda, AutenticacionUsuarioHacienda, EstadisticasUsuariosHacienda e InicioSesionUsuario observan la misma lista cargada una sola vez en Program. La alternativa de inyectar directamente List<Usuario> funcionaría técnicamente, pero dejaría el estado mutable sin un propietario explícito y debilitaría la separación lograda por H-20.

H-21 — VacunaService depende de Hacienda y PersistenciaService concretos

Problemas señalados en el hallazgo

VacunaService depende directamente de Hacienda y PersistenciaService aunque solo necesita capacidades específicas de inventario, consulta y actualización.

VacunaService es un módulo de aplicación que coordina casos de uso de vacunación, pero en el AS-IS conoce dos clases completas: Hacienda y PersistenciaService. El problema no es que use información del dominio o que necesite guardar; el problema es que declara esas necesidades mediante implementaciones concretas más amplias que las capacidades que realmente requiere.

La matriz AS-IS documenta ambas asociaciones por inyección y el análisis de dependencias confirma que VacunaService conserva Hacienda y PersistenciaService en campos. H-21 reemplaza esas dos referencias concretas por capacidades específicas sin mover todavía la lógica interna del servicio.

- H-01 trasladó creación/aplicación de vacunas a VacunaService; por eso H-21 se resuelve en esta frontera y no devolviendo lógica a Hacienda.
- H-02 ya aisló las reglas de vacunación por Res y el límite de lote; esas abstracciones se conservan.
- H-14 creó ICargaVacuna en singular para cargar UNA vacuna a partir de campos persistidos dentro de PersistenciaService. H-21 no sustituye esa interfaz.
- H-15 hizo que PersistenciaService dependa de abstracciones para almacenamiento/validación; H-21 trabaja en la dirección opuesta: evita que VacunaService conozca PersistenciaService concreto.
- H-16 ya ofrece IConsultaPotreros; H-21 la reutiliza en lugar de inventar otra interfaz equivalente.

Solución aprobada

Cambio detallado

Frontera TO-BE aprobada

VacunaService deja de recibir Hacienda y PersistenciaService. En su lugar recibe únicamente las capacidades que necesita:

- IInventarioVacunas: acceso al inventario de vacunas que Hacienda ya posee.
- IConsultaPotreros: búsqueda/listado de potreros, reutilizada desde H-16 para localizar el contexto donde se aplica la vacuna.
- ICargaVacunas: obtener el catálogo de vacunas almacenado.
- IActualizacionInventarioVacunas: guardar el inventario de vacunas.
- IActualizacionHistorialVacunacion: guardar las vacunas aplicadas asociadas a las reses/potreros.

Hacienda es el único proveedor de IInventarioVacunas porque es quien posee la colección.

PersistenciaService es el proveedor de las tres capacidades de carga/guardado porque conserva exactamente esas operaciones. VacunaService solamente consume los contratos; no implementa ninguno de ellos.

IInventarioVacunas y preservación del estado

IInventarioVacunas no crea una nueva colección. Hacienda conserva L_vacunas sin cambio y expone Vacunas como un alias a la misma instancia para satisfacer el contrato. Así se evita que el servicio reciba toda Hacienda y, al mismo tiempo, se preservan identidad, orden y mutabilidad actuales del inventario.

No se introducen métodos Agregar/Retirar nuevos en este hallazgo ni se cambia el encapsulamiento general de Hacienda. H-21 solo expresa mediante una abstracción la capacidad que VacunaService ya utilizaba.

Diferencia entre ICargaVacuna (H-14) e ICargaVacunas (H-21)

Contrato	Responsabilidad	Consumidor principal
ICargaVacuna (singular)	Construir/cargar una vacuna individual desde los campos ya leídos.	PersistenciaService
ICargaVacunas (plural)	Solicitar el catálogo/lista de vacunas almacenadas.	VacunaService

No son duplicadas: una pertenece al detalle interno de cómo se forma una vacuna durante la carga; la otra representa la capacidad de aplicación de obtener el catálogo sin conocer PersistenciaService.

Persistencia: capacidades estrechas, no la clase completa

Las operaciones existentes CargarVacunas(), GuardarVacunas(List<Vacuna>) y GuardarVacunasAplicadas(List<Potrero>) permanecen en PersistenciaService con las mismas firmas,

validaciones, mensajes y formatos. La diferencia es que ahora también satisfacen interfaces específicas. VacunaService declara los contratos, no la implementación concreta.

No se crea una interfaz genérica de persistencia ni un contrato con todas las operaciones de archivos. Eso volvería a acoplar el servicio a capacidades que no necesita y repetiría el problema con otra forma.

Matriz explícita de inversión de dependencias

Campo exigido	H-21
Módulo de alto nivel	VacunaService.
Módulo de bajo nivel / detalle concreto	Hacienda y PersistenciaService.
Abstracción que desacopla	IIventarioVacunas; IConsultaPotreros; ICargaVacunas; IActualizacionInventarioVacunas; IActualizacionHistorialVacunacion.
Composition root	p_mvcHacienda/Program.cs, método Main: expone Hacienda como IIventarioVacunas, ConsultaPotreros como IConsultaPotreros y la misma PersistenciaService como ICargaVacunas/IActualizacionInventarioVacunas/IActualizacionHistorialVacunacion.

Garantía de conducta observable

- No cambian los objetos Bacteriana/Viva creados ni sus parámetros.
- No cambian reglas de vacunación, cantidades, cálculos ni estadísticas.
- No cambia el orden de aplicación, alta/baja del inventario ni publicación de eventos.
- No cambian mensajes, excepciones ni cadenas de retorno.
- No cambian nombres, contenido ni formato de archivos.
- Las operaciones de PersistenciaService siguen siendo las mismas; solo se consumen a través de contratos.

Qué se gana e impacto para negocio y mantenimiento

El cambio reduce el tiempo y costo de sustitución o prueba: VacunaService puede probarse con implementaciones controladas de inventario, consulta y guardado sin construir toda Hacienda ni utilizar la persistencia real. Un cambio en la tecnología de almacenamiento queda detrás de las capacidades de persistencia y un cambio en la implementación del estado no obliga a reescribir el caso de uso de vacunación. El radio de regresión disminuye porque las pruebas pueden aislar cada colaboración.

H-22 — VacunaService conoce explícitamente Bacteriana y Viva

Problemas señalados en el hallazgo

VacunaService construye Bacteriana y Viva directamente y vuelve a distinguir esos tipos concretos para calcular estadísticas.

La solución no reemplaza esas comprobaciones por un enum, un switch o una cadena if/else equivalente. La arquitectura separa la creación específica y utiliza polimorfismo para una vacuna ya existente.

Solución aprobada

Cambio detallado

TipoVacuna: uso y criterio de industria

Se agrega en Vacuna una propiedad pública, de solo lectura y polimórfica llamada TipoVacuna. Este recurso es habitual en sistemas empresariales cuando una entidad necesita exponer su clasificación para reportes, serialización, persistencia o presentación. La práctica importante es no utilizar el discriminador como sustituto del polimorfismo para decidir comportamiento.

Por eso, en este diseño TipoVacuna únicamente informa la categoría de una Vacuna que ya existe. Bacteriana y Viva sobrescriben la propiedad y VacunaService puede clasificar para estadísticas sin preguntar “vacuna is Bacteriana” o “vacuna is Viva”. No se introduce un enum para seleccionar conducta. Cuando el conjunto de valores es realmente cerrado, en industria también son frecuentes enum o Value Objects; en este proyecto se mantiene una propiedad de negocio extensible porque el objetivo explícito es poder incorporar nuevos tipos sin reabrir el servicio.

Elemento	Responsabilidad en H-22
Vacuna.TipoVacuna	Contrato común: toda vacuna informa su tipo.
Bacteriana.TipoVacuna	Entrega la etiqueta actual de la categoría bacteriana.
Viva.TipoVacuna	Entrega la etiqueta actual de la categoría viva.
VacunaService.ObtenerEstadisticas()	Agrupar/contabiliza por TipoVacuna sin conocer las clases concretas.

Los valores/etiquetas existentes se conservan de modo que las claves, cantidades y orden actualmente observables en las estadísticas no cambien. TipoVacuna no se usa para construir objetos ni para escoger qué algoritmo ejecutar.

Por qué la creación se separa en dos fronteras

La creación no puede resolverse de la misma forma que las estadísticas porque antes de crear el objeto todavía no existe una Vacuna que pueda responder TipoVacuna. Además, Bacteriana y Viva requieren datos específicos distintos: la primera usa periodo de aplicación y la segunda grado de atenuación. Un método genérico con un “datoEspecifico” o parámetros opcionales mantendría la misma decisión escondida y debilitaría OCP.

Se crean dos responsabilidades de negocio sencillas: CreacionVacunaBacteriana y CreacionVacunaViva. Cada una conoce solamente el subtipo que crea y conserva íntegramente sus reglas, mensajes, incorporación al inventario y guardado. No se identifica ni documenta ningún patrón de diseño; son clases de responsabilidad específica apoyadas en interfaces estrechas.

Abstracción	Implementación	Qué conserva
ICreacionVacunaBacteriana	CreacionVacunaBacteriana	Las dos firmas bacterianas existentes: creación individual y por cantidad; periodo_aplicacion conserva su significado.
ICreacionVacunaViva	CreacionVacunaViva	Las dos firmas vivas existentes: creación individual y por cantidad; grado_atenuacion conserva su significado.

La interfaz original ICreacionVacuna agrupaba cuatro sobrecargas que representaban dos capacidades distintas. En el TO-BE se reemplaza por dos interfaces estrechas porque ningún creador debe verse obligado

a conocer los parámetros del otro tipo. Esto también sigue la recomendación docente de preferir capacidades específicas frente a contratos sobrecargados.

Reutilización de H-21 y H-02

- `IIventarioVacunas` se reutiliza para que ambas clases incorporen las vacunas a la misma colección que ya pertenece a Hacienda; no se crea otro inventario.
- `IActualizacionInventarioVacunas` se reutiliza para conservar exactamente la misma persistencia después de crear.
- `ReglaLoteVacunas` de H-02 se mantiene para las variantes por cantidad y conserva el mismo límite de lote.
- `CreacionVacunaBacteriana` es la única frontera nueva que conoce el constructor de Bacteriana; `CreacionVacunaViva` es la única que conoce el constructor de Viva.
- Program continúa como composition root y suministra las implementaciones a los consumidores.

Preservación estricta del comportamiento

Aspecto	Preservación H-22
Creación individual	Se conservan las mismas firmas específicas, parámetros, validaciones, constructores efectivos y mensajes.
Creación por cantidad	Se conservan cantidad, formación de lotes, límite de <code>ReglaLoteVacunas</code> y orden de incorporación/guardado.
Inventario	Se modifica la misma colección de Hacienda proporcionada por <code>IIventarioVacunas</code> ; no se copia ni reemplaza.
Persistencia	Se mantiene <code>IActualizacionInventarioVacunas</code> y, por debajo, la misma implementación aprobada en H-21/H-15.
Estadísticas	Se conservan las mismas claves, cantidades y orden; solo cambia la forma interna de clasificar cada Vacuna.
Mensajes y excepciones	No se cambian textos, condiciones, orden de validación ni propagación de excepciones.

Por qué cumple OCP

Después de H-22, `VacunaService` ya no necesita modificarse para conocer una nueva clase concreta de vacuna en creación ni para agregar una nueva comprobación “is” en estadísticas. Una nueva vacuna puede aportar su propio `TipoVacuna` y su propia frontera de creación específica. Las clases `CreacionVacunaBacteriana` y `CreacionVacunaViva` tampoco se modifican porque cada una continúa cerrada alrededor de su tipo actual.

No se traslada el problema a una comparación de cadenas dentro de `VacunaService`: el servicio no pregunta si `TipoVacuna` es “Bacteriana” o “Viva” para ejecutar comportamiento; únicamente utiliza el valor polimórfico para clasificación estadística.

Herencia conservada Vacuna → Bacteriana/Viva

Criterio	Justificación H-22
Por qué herencia	Bacteriana y Viva siguen siendo Vacuna y comparten identidad, nombre, lote, fechas y contrato <code>TipoVacuna</code> .
Por qué no composición	Componer una Vacuna dentro de cada subtipo duplicaría identidad y obligaría a delegar miembros que ya pertenecen naturalmente a la misma entidad.
Precondiciones	<code>TipoVacuna</code> puede consultarse sobre cualquier Vacuna

Criterio	Justificación H-22
	válida sin una precondition adicional.
Postcondiciones	Cada subtipo informa consistentemente su categoría actual y las operaciones heredadas conservan sus resultados.
Invariantes	El TipoVacuna de una instancia permanece coherente con su subtipo durante toda su vida.
Excepciones	El getter TipoVacuna no introduce excepciones nuevas; las excepciones de construcción permanecen donde ya existían.
Sustituibilidad	Bacteriana y Viva pueden tratarse como Vacuna y responder TipoVacuna sin comprobación de clase concreta.

Qué se gana e impacto para negocio y mantenimiento

Dimensión	Beneficio
Tiempo	Agregar un nuevo tipo deja de exigir editar creación y estadísticas dentro de VacunaService.
Costo	Se reduce el número de componentes existentes que deben modificarse y volver a probarse ante una extensión.
Riesgo	Disminuye la posibilidad de romper creación o conteos de Bacteriana/Viva al incorporar una categoría nueva.
Mantenibilidad	PeriodoAplicacion y Atenuacion permanecen en fronteras donde conservan un significado único, evitando parámetros ambiguos.
Evolución	Cada nueva categoría puede añadir su propio TipoVacuna y su propia capacidad de creación sin reabrir las clases de creación actuales.

H-23 — VacunaService concentra aplicación, consultas, inventario, persistencia, mensajes y estadísticas

Problemas señalados en el hallazgo

VacunaService concentra aplicación de vacunas, inventario, consultas, estadísticas, notificaciones, composición de mensajes y registro del resultado.

- H-21 ya eliminó las dependencias directas de VacunaService hacia Hacienda y PersistenciaService. Sus capacidades no se pierden: se redistribuyen entre las nuevas fronteras que realmente las necesitan.
- H-22 ya extrajo la creación concreta a CreacionVacunaBacteriana y CreacionVacunaViva. Por tanto H-23 no vuelve a crear ni seleccionar Bacteriana/Viva.
- H-22 también dejó ObtenerEstadisticas() usando Vacuna.TipoVacuna, por lo que EstadisticasVacunas conserva esa solución polimórfica sin enum, switch ni if/else por tipo.
- H-02 mantiene IReglasVacunacionRes y las reglas sanitarias por tipo de res. AplicacionVacuna continúa utilizándolas; H-23 no modifica límites ni esquema sanitario.
- H-15 mantiene separado el almacenamiento físico mediante IAlmacenamientoHacienda; ninguna de las clases nuevas conoce File, Directory o Path.

Solución aprobada

Cambio detallado

División aprobada: seis responsabilidades

Clase	Única responsabilidad
AplicacionVacuna	Orquestar el caso de uso de vacunar: localizar contexto, ejecutar las reglas existentes y solicitar a los colaboradores especializados sus tareas en el mismo orden.
InventarioVacunas	Localizar y modificar exclusivamente el inventario lógico de vacunas que ya pertenece a Hacienda.
ConsultaVacunas	Responder consultas de vacunas disponibles y vacunas aplicadas, sin modificar estado.
EstadisticasVacunas	Calcular las estadísticas de vacunas a partir de la información consultada y de Vacuna.TipoVacuna.
NotificacionVacunacion	Coordinar publishers y composición/formato de los mensajes generados en el flujo de vacunación.
RegistroVacunacion	Coordinar exclusivamente que el estado resultante quede reflejado en inventario e historial mediante IActualizacionInventarioVacunas e IActualizacionHistorialVacunacion.

Por qué RegistroVacunacion no duplica PersistenciaService ni almacenamiento

RegistroVacunacion pertenece al caso de uso y responde “qué resultados debo registrar después de vacunar y en qué orden”. No serializa ni escribe archivos. Consume IActualizacionInventarioVacunas e IActualizacionHistorialVacunacion.

PersistenciaService conserva la lógica existente de convertir/coordinar los datos persistidos y sus validaciones. Por debajo, IAlmacenamientoHacienda y AlmacenamientoArchivosHacienda siguen respondiendo “cómo y dónde se almacenan físicamente”. Cambiar de archivos a otra tecnología no obliga a modificar AplicacionVacuna ni RegistroVacunacion.

Flujo TO-BE y preservación del orden

El flujo observable permanece: localizar potrero/res y vacuna; validar; aplicar la vacuna; actualizar el mismo inventario; ejecutar las mismas notificaciones; registrar inventario e historial con las mismas operaciones subyacentes; y devolver el mismo mensaje. La división no autoriza reordenar pasos.

AplicacionVacuna conserva las dos operaciones AplicarVacuna existentes. La realización de IVacunacion se mueve desde VacunaService hacia AplicacionVacuna porque ahora esa es la clase que proporciona realmente esa capacidad.

Responsabilidad de InventarioVacunas

InventarioVacunas no crea una segunda lista. Recibe IInventarioVacunas y trabaja sobre la misma colección de Hacienda. La búsqueda por lote y la modificación/retirada que antes estaban dentro del flujo de VacunaService quedan localizadas aquí. Las reglas actuales de selección, orden y modificación se conservan sin agregar una política nueva.

CreacionVacunaBacteriana y CreacionVacunaViva también continúan usando IActualizacionInventarioVacunas para reflejar el inventario después de crear. Esto no convierte InventarioVacunas en persistencia: una cosa es modificar el estado lógico y otra registrar físicamente ese estado.

Consultas y estadísticas

ObtenerVacunasDisponibles() y ObtenerVacunasAplicadas(potreroId, nombreRes) se trasladan íntegramente a ConsultaVacunas. Esta clase consume ICargaVacunas y reutiliza IConsultaPotreros cuando la consulta necesita recorrer el estado de potreros/reses.

ObtenerEstadisticas() se traslada a EstadisticasVacunas. La clase no conoce Bacteriana ni Viva: conserva TipoVacuna aprobado en H-22 y se limita al cálculo. Un cambio en un reporte deja de poner en riesgo la ejecución de la vacunación.

Notificaciones y mensajes

Los campos publisher_vacunacion_completa y publisher_vacuna_vencida salen de VacunaService y pasan a NotificacionVacunacion. También se trasladan ConsolidarValidaciones(...) y AsegurarPuntoFinal(...). Los publishers no cambian, sus reglas internas no cambian y se conserva la misma secuencia de invocación y composición de textos. H-27 seguirá tratando por separado las comprobaciones concretas que existan dentro de PublisherVacunacionCompletada.

Interfaces y semántica proveedor/consumidor

Contrato	Quién lo proporciona	Quién lo consume después de H-23
IIventarioVacunas	Hacienda	InventarioVacunas y las fronteras de creación que necesitan el mismo estado.
ICargaVacunas	PersistenciaService	ConsultaVacunas.
IActualizacionInventarioVacunas	PersistenciaService	RegistroVacunacion y CreacionVacunaBacteriana/CreacionVacunaViva.
IActualizacionHistorialVacunacion	PersistenciaService	RegistroVacunacion.
IConsultaPotreros	ConsultaPotreros	AplicacionVacuna y ConsultaVacunas.
IVacunacion	AplicacionVacuna	Consumidores que necesitan ejecutar la vacunación mediante la capacidad existente.

Ninguna de estas relaciones significa que las clases consumidoras compartan responsabilidad o herencia. Las interfaces expresan capacidades proporcionadas por un componente y necesidades consumidas por otro, manteniendo la distinción corregida desde H-16.

Program como composition root

Program continúa siendo el único punto autorizado a conocer y conectar las implementaciones concretas. Construye InventarioVacunas, ConsultaVacunas, EstadisticasVacunas, NotificacionVacunacion, RegistroVacunacion y AplicacionVacuna con las mismas instancias compartidas necesarias. Esto no se presenta como patrón de diseño; es el composition root exigido para documentar la composición.

Cambios en VacunaController sin cerrar H-29

Al desaparecer la clase multipropósito VacunaService, VacunaController deja de mantener _vacunaService y recibe AplicacionVacuna, ConsultaVacunas y EstadisticasVacunas para las operaciones que antes obtenía de aquella clase. Las capacidades ICreacionVacunaBacteriana e ICreacionVacunaViva creadas en H-22 permanecen sin cambio.

La selección que VacunaController todavía hace a partir de tipoVacuna sigue expresamente pendiente de H-29. H-23 no reemplaza esa decisión por enum, switch ni otra cadena equivalente y no declara resuelto el OCP del controlador.

Preservación estricta del comportamiento observable

Aspecto	Garantía H-23
Reglas sanitarias	Sin cambios; se reutilizan las mismas reglas H-02 y validaciones existentes.
Inventario	Misma colección de Hacienda, misma búsqueda/modificación y mismo orden.
Vacunas aplicadas	Misma asociación con las reses y mismo historial resultante.
Persistencia	Mismos GuardarVacunas/GuardarVacunasAplicadas por debajo de los contratos renombrados; mismos archivos y formatos.
Notificaciones	Mismos publishers, mismo orden y mismas cadenas.
Consultas	Mismos parámetros y colecciones devueltas.
Estadísticas	Mismas claves/valores/orden; se conserva TipoVacuna de H-22.
Excepciones	No se agregan, eliminan ni reordenan excepciones observables.

Qué se gana e impacto para negocio y mantenimiento

Dimensión	Impacto esperado
Tiempo de cambio	Un ajuste de estadísticas, consulta o formato de mensaje se realiza en su frontera sin revisar toda la lógica de aplicación.
Costo de pruebas	Las pruebas pueden centrarse en la responsabilidad modificada; el caso de uso completo solo requiere regresión cuando cambia su coordinación.
Riesgo	Disminuye la probabilidad de que una modificación de reporte, persistencia o mensajes altere accidentalmente las reglas sanitarias de vacunación.
Evolución	Nuevas necesidades de consulta, reporte o almacenamiento pueden evolucionar detrás de las fronteras existentes sin reconstruir VacunaService.

H-24 — VentaService depende directamente de Hacienda y de un PersistenciaService innecesario

Problemas señalados en el hallazgo

VentaService depende de Hacienda completa para acceder al estado necesario y recibe PersistenciaService aunque esta dependencia no se utiliza.

Solución aprobada

Cambio detallado

Decisión clave: la dependencia no utilizada se elimina, no se abstrae

PersistenciaService se retira por completo del constructor y de los campos de VentaService. No se crea IPersistenciaVentas ni otra interfaz equivalente porque no existe una colaboración que conservar: la

dependencia estaba inyectada pero no se utilizaba. Crear una abstracción para una dependencia inexistente mantendría complejidad sin beneficio y sería contrario al criterio aplicado desde H-03 y H-17.

Por tanto, H-24 no agrega ninguna capacidad de guardado/persistencia para VentaService. Si en otro hallazgo o evolución futura aparece una necesidad real de persistencia, deberá modelarse desde esa necesidad concreta y no anticiparse artificialmente aquí.

Abstracciones que sí corresponden a necesidades reales

Abstracción	Proveedor	Consumidor	Responsabilidad
IEstadoVentas	Hacienda	VentaService	Entregar una vista de solo lectura de las ventas existentes.
IRegistroVenta	Hacienda	VentaService	Incorporar una Venta al mismo estado compartido que ya conserva Hacienda.
IConsultaPotreros	ConsultaPotreros	VentaService	Localizar/consultar potreros mediante la frontera ya aprobada en H-16.

IEstadoVentas e IRegistroVenta separan lectura y modificación porque son capacidades distintas. Hacienda continúa siendo la dueña real de l_ventas; no se crea una colección paralela. IConsultaPotreros no se duplica: se reutiliza exactamente la abstracción introducida en H-16.

Responsabilidad y semántica proveedor/consumidor

Hacienda implementa IEstadoVentas e IRegistroVenta porque es quien posee el estado de ventas. VentaService no implementa esas interfaces: las consume. ConsultaPotreros implementa IConsultaPotreros y VentaService la consume. Esto evita repetir el error conceptual de confundir a quien proporciona una capacidad con quien la necesita.

No existe herencia nueva entre Hacienda, VentaService o ConsultaPotreros. Las relaciones son realizaciones de interfaces en los proveedores y asociaciones/dependencias hacia abstracciones en el consumidor.

Estructura TO-BE

VentaService deja de mantener _hacienda y _persistencia. En su lugar mantiene _estadoVentas : IEstadoVentas, _registroVenta : IRegistroVenta y _consultaPotreros : IConsultaPotreros. El constructor recibe únicamente esas tres capacidades.

Las operaciones ObtenerTodasLasVentas(), ObtenerVentasPorPotrero(...), ObtenerVentasPorFechas(...), ObtenerEstadisticas() y VenderRes(...) permanecen en VentaService con sus mismas firmas y propósito. H-24 cambia cómo obtiene el estado necesario, no el comportamiento comercial.

Matriz explícita de inversión de dependencias

Campo exigido	H-24
Módulo de alto nivel	VentaService.
Módulo de bajo nivel / detalle concreto	Hacienda y PersistenciaService.
Abstracción que desacopla	IEstadoVentas, IRegistroVenta e IConsultaPotreros. PersistenciaService no recibe una abstracción sustituta porque su dependencia era innecesaria y se elimina.

Campo exigido	H-24
Composition root	p_mvcHacienda/Program.cs, método Main: expone la misma Hacienda como IEstadoVentas/IRegistroVenta, registra ConsultaPotreros como IConsultaPotreros y resuelve VentaService contra esas abstracciones.

Preservación del comportamiento observable

Aspecto	Garantía H-24
Colección de ventas	Es exactamente la misma l_ventas de Hacienda; IEstadoVentas solo expone una vista de lectura.
Registro de una venta	IRegistroVenta incorpora la misma Venta al mismo estado compartido.
Localización de potreros	Se reutiliza IConsultaPotreros, que finalmente consulta el mismo estado de potreros de Hacienda.
Consultas y filtros	Mismos criterios, parámetros y resultados de ObtenerTodasLasVentas, ObtenerVentasPorPotrero y ObtenerVentasPorFechas.
Estadísticas	Mismos cálculos, claves, valores y orden actuales.
Venta	Mismos identificadores, monto, reglas, creación de Venta, cambios sobre la res/potrero, mensajes y excepciones.
PersistenciaService	Su retiro no altera la conducta porque no era utilizado por VentaService.

Por qué se conservan Venta, Potrero y Res como tipos de dominio

H-24 no pretende convertir todo tipo concreto en una interfaz. VentaService pertenece al caso de uso de ventas y es correcto que sus operaciones manejen entidades del dominio. La inversión se aplica en la frontera hacia el dueño concreto del estado y hacia una dependencia técnica innecesaria, no sobre cada entidad con la que el caso de uso trabaja.

La dependencia VentaService → Potrero documentada previamente para ObtenerVentasPorPotrero también se conserva: la consulta accede a la identificación del Potrero contenido en cada Venta. Esa relación es de dominio y no contradice DIP.

Impacto de hallazgos anteriores

- H-01 mantiene VentaService como responsable del caso de uso de venta; H-24 no devuelve esa lógica a Hacienda.
- H-16 aporta IConsultaPotreros y ConsultaPotreros, que ahora se reutilizan para no entregar a VentaService toda Hacienda.
- H-17 confirmó el mismo criterio: una dependencia concreta no utilizada se elimina en lugar de envolverla en una interfaz artificial.
- H-15 sigue siendo la frontera de almacenamiento físico. Como VentaService no utilizaba PersistenciaService, H-24 no introduce relación con IAlmacenamientoHacienda.

Qué se gana e impacto para negocio y mantenimiento

Dimensión	Impacto esperado
-----------	------------------

Dimensión	Impacto esperado
Tiempo de cambio	Cambiar la implementación que conserva el estado de ventas o la forma de consultar potreros no obliga a reescribir VentaService mientras se mantengan los contratos.
Costo de pruebas	VentaService puede probarse con colaboradores que cumplan tres contratos pequeños; ya no requiere construir una Hacienda completa ni un PersistenciaService inútil.
Riesgo	Se reduce el radio de impacto de cambios en el dueño del estado y se elimina una dependencia que podía inducir uso accidental de persistencia dentro del servicio.
Mantenimiento	El constructor expresa las necesidades reales del servicio y deja de comunicar una colaboración inexistente con PersistenciaService.

H-25 y H-26 — Publishers de peso conocen tipos concretos de Res

Problemas señalados en el hallazgo

PublisherPesoMin y PublisherPesoVenta distinguen explícitamente Ternero, Cebon y Novillo para obtener umbrales de peso, por lo que cada nueva categoría obliga a modificar ambos publishers.

Solución aprobada

Cambio detallado

Frontera elegida: IReglaPesoRes

IReglaPesoRes reúne los dos valores que pertenecen a la misma política de peso por categoría: el peso mínimo y el peso de venta. Los publishers consultan esta abstracción a través de la Res y dejan de distinguir directamente entre Ternero, Cebon y Novillo.

PesoMinimo y PesoVenta se mantienen juntos porque pertenecen al mismo eje de política de peso por categoría. No se incluyen edad, vacunación, incorporación a potrero ni otras reglas que cambian por motivos diferentes.

Reglas concretas y valores preservados

Regla	Peso mínimo	Peso de venta
ReglaPesoTernero	150	250
ReglaPesoCebon	290	420
ReglaPesoNovillo	400	550

Estos valores ya existían en ReglaRes. H-25/H-26 no inventan ni ajustan ningún umbral: únicamente trasladan la variación a clases que pueden extenderse sin modificar los publishers.

Cómo llega la regla a cada Res

Program configura las reglas de peso concretas de Ternero, Cebon y Novillo. Cada ReglaTipoPotrero recibe la política que corresponde a su categoría y la suministra a la Res durante su construcción. La entidad conserva esa IReglaPesoRes como una referencia de solo lectura, por lo que los publishers pueden consultar los umbrales sin realizar selección por tipo concreto.

No se crea ReglaPesoTernero dentro de Ternero ni se selecciona con switch, enum o comprobaciones de tipo. Program es quien conoce los concretos; la Res conserva la política que corresponde a su categoría.

H-25 — PublisherPesoMin

PublisherPesoMin deja de preguntar si la res concreta es Ternero, Cebon o Novillo para escoger los valores 150, 290 o 400. En el TO-BE obtiene el umbral mínimo desde la política IReglaPesoRes asociada a la propia Res. Se conservan exactamente los mismos valores, pero desaparece la enumeración de subtipos dentro del publisher.

El publisher sigue evaluando y notificando exactamente la misma condición. Solo deja de conocer las tres clases concretas. Una categoría futura aporta su IReglaPesoRes y PublisherPesoMin no cambia.

H-26 — PublisherPesoVenta

PublisherPesoVenta aplica el mismo criterio. Antes distinguía entre Ternero, Cebon y Novillo para seleccionar respectivamente los umbrales 250, 420 y 550; después consulta el peso de venta suministrado por IReglaPesoRes. Los valores y el momento de notificación no cambian, pero el publisher queda abierto a nuevas categorías.

H-26 reutiliza exactamente la misma abstracción de H-25. No se crea una segunda familia de reglas porque peso mínimo y peso de venta son dos valores de la misma política de peso por categoría.

Por qué no usar TipoRes, enum o un selector central

TipoRes es útil para clasificación y estadísticas, pero usarlo aquí para preguntar «si TipoRes es Ternero...» volvería a introducir una decisión central cerrada. Un enum tendría el mismo problema. IReglaPesoRes permite extender la política agregando una nueva implementación sin modificar PublisherPesoMin ni PublisherPesoVenta.

Por qué la política de peso no se mezcla con ReglasEdadRes

Edad y peso pueden cambiar por decisiones de negocio distintas. Una modificación de los límites de edad no implica modificar los pesos mínimo o de venta. Mantener ReglasPesoRes como grupo separado conserva SRP y evita que una clase por categoría se convierta en un contenedor de todas las reglas de Ternero/Cebon/Novillo.

Preservación de comportamiento observable

- Los seis valores de peso permanecen exactamente iguales.
- Los publishers conservan sus eventos, mensajes y momento de notificación.
- No cambia cómo se modifica Peso en Res.
- No se agrega enum, switch, if por subtipo ni patrón de selección.
- El único cambio es estructural: la obtención del umbral se delega a IReglaPesoRes.

Qué se gana e impacto para negocio y mantenimiento

Cambio futuro	Antes	TO-BE	Beneficio
Nueva categoría de Res	modificar ambos publishers	agregar una nueva ReglaPesoRes y composición	menos código existente modificado y menor regresión
Cambiar peso mínimo de Novillo	editar una regla general/ramas	modificar ReglaPesoNovillo	pruebas localizadas
Cambiar peso de venta de Cebon	riesgo de afectar otras categorías	modificar ReglaPesoCebon	menor tiempo de cambio

H-27 — PublisherVacunacionCompletada discrimina tipos concretos de Res

Problemas señalados en el hallazgo

PublisherVacunacionCompletada distingue tipos concretos de Res para decidir si el esquema de vacunación está completo.

Solución aprobada

Cambio detallado

Por qué no se crea otra interfaz

IReglasVacunacionRes ya expresa exactamente la capacidad requerida: validar límites de vacunas y determinar si el esquema está completo. Crear una interfaz específica para el publisher duplicaría el mismo eje de variación y generaría dos fuentes de verdad para la política sanitaria.

PublisherVacunacionCompletada utiliza la capacidad IReglasVacunacionRes para determinar si el esquema sanitario está completo. Ternero, Cebon y Novillo proporcionan esa capacidad, de modo que el publisher se limita a detectar y notificar mientras la decisión sanitaria permanece en la regla correspondiente a cada res.

Impacto para negocio y mantenimiento

Agregar una nueva categoría de res ya no exige abrir un publisher probado ni volver a modificar las ramas de las categorías existentes. Se reduce el tiempo de cambio y el riesgo de alterar notificaciones sanitarias vigentes.

H-28 — La jerarquía Validacion no cumple sustitución segura

Problemas señalados en el hallazgo

La jerarquía Validacion obliga a sus subclasses a ofrecer operaciones que no soportan y que terminan en NotImplementedException, por lo que los subtipos no pueden sustituir con seguridad a la base.

H-28 se cierra formalmente sin introducir cambios estructurales nuevos porque la causa ya fue eliminada en H-12. La antigua superclase Validacion obligaba a los validadores concretos a prometer operaciones que no podían ejecutar, provocando NotImplementedException. Esa jerarquía fue eliminada y reemplazada por IValidador<T>.

La jerarquía original Validacion obligaba a los validadores concretos a exponer operaciones para varios dominios, aunque cada uno solo podía ejecutar una de ellas. El TO-BE elimina esa superclase y utiliza IValidador<T>: ValidadorRes, ValidadorPotrero, ValidadorVacuna y ValidadorVenta realizan el contrato únicamente para el tipo que realmente soportan. Así desaparecen las operaciones incompatibles y las excepciones de funcionalidad no implementada.

Solución aprobada

Cambio detallado

Por qué la herencia se elimina y no se conserva

No existía un comportamiento común completo que los cuatro hijos pudieran sustituir con seguridad. Conservar Validacion y dejar métodos vacíos o NotImplementedException mantendría la violación LSP. La decisión correcta es no conservar una herencia que no representa sustituibilidad real.

Verificación LSP exigida por Fase 3	Decisión H-28
Herencia vs composición/interfaz	Se elimina la herencia; cada validador realiza IValidador<T> para el tipo que realmente soporta.
Precondiciones	Cada contrato acepta únicamente su T correspondiente; no se fortalecen condiciones ocultas para otros tipos.
Postcondiciones	Validar(T) conserva el resultado de la validación efectiva existente.
Invariantes	Ningún validador queda obligado a soportar dominios ajenos.
Excepciones	Desaparecen NotImplementedException de operaciones incompatibles; se conservan las excepciones reales de cada validación.
Sustituibilidad	Cada implementación puede usarse donde se requiere IValidador<T> sin encontrar operaciones prometidas pero imposibles.

Qué se gana e impacto para negocio y mantenimiento

Los consumidores ya no pueden invocar por contrato una validación que la implementación no soporta. Esto reduce fallos en ejecución, simplifica pruebas y permite incorporar nuevos validadores sin ampliar una jerarquía artificial.

H-29 — VacunaController selecciona explícitamente Bacteriana o Viva

Problemas señalados en el hallazgo

VacunaController selecciona explícitamente entre Bacteriana y Viva e interpreta parámetros específicos de cada tipo dentro del controlador.

Solución aprobada

Cambio detallado

VacunaController deja de conocer los detalles particulares de Bacteriana y Viva y consume ICreacionPorTipoVacuna. FormularioBacteriana y FormularioViva proporcionan esa capacidad en la capa MVC y, a su vez, delegan en ICreacionVacunaBacteriana e ICreacionVacunaViva. Las clases de creación de negocio continúan siendo las responsables de producir las entidades Bacteriana y Viva.

Los formularios se ubican en la capa MVC. No reemplazan las entidades Bacteriana/Viva ni las clases de creación de H-22. Su única responsabilidad es interpretar los datos recibidos desde presentación y delegar la creación al contrato específico ya existente.

Por qué no se mete esta responsabilidad en CreacionVacunaBacteriana/Viva

CreacionVacunaBacteriana y CreacionVacunaViva representan la creación de negocio y ya fueron separadas en H-22 porque sus parámetros tienen significados diferentes. Hacer que esas clases interpreten también el formulario MVC les daría una segunda razón de cambio: cambiarían tanto cuando cambia la creación de negocio como cuando cambia la forma en que la interfaz captura o entrega los datos.

FormularioBacteriana

= entiende datos de presentación:

nombre, lote, fechas, periodoAplicacion

CreacionVacunaBacteriana

= ejecuta la creación de negocio usando datos ya interpretados

Bacteriana

= entidad del dominio

Cambiar el formulario NO debe obligar a modificar la creación de negocio. Cambiar la creación de negocio NO debe obligar a modificar el formulario.

Por qué sí se relacionan con Bacteriana y Viva existentes

No se crean nuevas variantes del dominio. FormularioBacteriana usa ICreacionVacunaBacteriana, cuya implementación sigue creando la misma Bacteriana existente. FormularioViva hace lo mismo con ICreacionVacunaViva y la misma clase Viva. Las nuevas clases son únicamente la frontera de presentación necesaria para que VacunaController deje de conocer los detalles de cada variante.

Por qué no se usa datoEspecifico ni una firma falsa común

PeriodoAplicacion y Atenuacion no representan el mismo concepto. Por eso no se introduce un parámetro datoEspecifico ni se obliga a Bacteriana a recibir Atenuacion o a Viva a recibir PeriodoAplicacion.

ICreacionPorTipoVacuna recibe los datos del formulario como conjunto de entrada de presentación; cada Formulario interpreta únicamente los campos que realmente le corresponden y luego llama a su creación específica.

Cómo desaparece el if sin usar enum o switch

Program configura las implementaciones por TipoVacuna. VacunaController obtiene la capacidad correspondiente y ejecuta Crear. El algoritmo del controlador no contiene nombres concretos de vacunas.

Program relaciona los valores actuales Bacteriana y Viva con FormularioBacteriana y FormularioViva. VacunaController recibe el tipo desde la presentación, solicita la implementación de ICreacionPorTipoVacuna que corresponde y delega la interpretación de los datos. Para una vacuna futura se agregaría un nuevo formulario, su creación específica y su registro en Program, sin modificar el controlador.

Matriz explícita de inversión de dependencias

Campo exigido	H-29
Módulo de alto nivel	VacunaController: coordina la entrada/salida MVC y selecciona una capacidad de creación sin conocer la clase concreta del formulario.
Módulo de bajo nivel / detalle concreto	FormularioBacteriana y FormularioViva, que interpretan los datos específicos de presentación y delegan en las capacidades de creación de negocio.
Abstracción que desacopla	ICreacionPorTipoVacuna; el controlador consume además un IReadOnlyDictionary<string, ICreacionPorTipoVacuna> y una implementación predeterminada del mismo contrato para preservar el else histórico.

Composition root	p_mvcHacienda/Program.cs, método Main: registra FormularioBacteriana/FormularioViva, construye el diccionario de ICreacionPorTipoVacuna y registra FormularioViva como implementación predeterminada; el contenedor lo suministra a VacunaController.
-------------------------	---

Preservación del else actual

El código AS-IS trata cualquier valor distinto de Bacteriana como Viva. Aunque esa conducta pueda ser cuestionable, corregirla ahora cambiaría el comportamiento observable y violaría la restricción del cliente. Por eso Program configura FormularioViva también como formulario predeterminado: si TipoVacuna no se encuentra, se conserva exactamente el mismo resultado que producía el else.

Por qué FormularioBacteriana/FormularioViva y no IngresoBacteriana/IngresoViva

Se eligió 'Formulario' porque expresa de manera intuitiva que estas clases pertenecen a la entrada MVC y representan la interpretación de los datos capturados para cada tipo. 'Ingreso' era ambiguo y podía confundirse con incorporación al inventario o con una operación de negocio.

Qué se gana e impacto para negocio y mantenimiento

Cambio futuro	Antes	Después	Beneficio
Nueva vacuna	modificar VacunaController y agregar otra rama	agregar formulario + creación específica + composición	menor tiempo y regresión
Cambia formulario de Viva	modificar código de controlador mezclado con otras variantes	modificar FormularioViva	riesgo localizado
Cambia creación de Bacteriana	riesgo de afectar entrada MVC	modificar CreacionVacunaBacteriana	separación presentación/negocio

H-30 — PotreroController conoce Hacienda, PersistenciaService y repite el guardado

Problemas señalados en el hallazgo

PotreroController conoce Hacienda y PersistenciaService y repite una operación de guardado que ya forma parte del caso de uso coordinado por PotreroService.

Solución aprobada

Cambio detallado

Frontera resultante

PotreroController conserva únicamente dependencias de presentación y aplicación: PotreroService para los casos de uso, IConsultaPotreros para las consultas y EstadisticasPotreros para los cálculos requeridos por las vistas. Hacienda, PersistenciaService y las llamadas directas de guardado desaparecen del controlador porque no pertenecen a su responsabilidad MVC.

PotreroController vuelve a tener una responsabilidad de presentación: recibir la petición HTTP, solicitar la operación o consulta correspondiente y producir la misma respuesta MVC. No administra el estado de Hacienda ni decide cuándo escribir físicamente los datos.

Por qué PersistenciaService se elimina y no se reemplaza por otra interfaz

Aquí una interfaz de persistencia en el controlador sería una abstracción incorrecta. El problema no es que el controller conozca una implementación concreta de guardado; el problema es que está intentando guardar. El caso de uso CrearPotrero ya coordina registro y actualización. Por ello la frontera correcta consiste en retirar la responsabilidad completa del controlador, no esconderla detrás de IActualizacionPotreros o IActualizacionReses.

Esta decisión no establece la regla general 'dependencia no usada = eliminar'. En cada hallazgo se pregunta primero si la capacidad pertenece conceptualmente a esa clase y si puede ser relevante para evolución. En H-30, persistir no pertenece a la presentación, por lo que mantener una dependencia futura de guardado en el controller debilitaría la frontera.

Por qué se elimina la segunda escritura sin cambiar el comportamiento

La segunda llamada de guardado ocurre después de que PotreroService.CrearPotrero ya ha ejecutado el guardado del mismo caso de uso. Por tanto se retira la coordinación redundante, pero no el comportamiento requerido: sigue existiendo una escritura, el potrero sigue registrado, el archivo final conserva el mismo estado y la respuesta web permanece igual.

Antes de H-30, el controlador solicitaba a PotreroService la creación del potrero y, después de que el propio servicio ya había registrado y guardado el resultado, ejecutaba una segunda escritura directa mediante PersistenciaService. En el TO-BE, el controlador delega una sola vez en PotreroService y es el caso de uso quien conserva el registro y el guardado necesarios. Se elimina la coordinación duplicada, no el efecto persistente.

Responsabilidades después de H-30

Elemento	Responsabilidad
PotreroController	Entrada/salida MVC; delegar creación, consultas y estadísticas.
PotreroService	Coordinar creación de potreros y modificación del estado correspondiente.
ConsultaPotreros / IConsultaPotreros	Consultar los potreros existentes.
EstadisticasPotreros	Calcular la información estadística.
PersistenciaService	Persistir/cargar datos a través de las abstracciones definidas en H-15.

Impacto para negocio y mantenimiento

Un cambio en almacenamiento deja de obligar a modificar o volver a probar el controlador web. Además, retirar la escritura duplicada reduce trabajo innecesario de E/S y disminuye el riesgo de inconsistencias si una de dos escrituras equivalentes falla. El cambio se traduce en menor tiempo de ejecución, menor radio de regresión y una frontera de presentación más estable.

H-32 — ResController conoce Hacienda y PersistenciaService

Problemas señalados en el hallazgo

ResController conoce Hacienda y PersistenciaService y coordina directamente persistencia dentro de los flujos de alimentación y venta.

Situación AS-IS	Decisión TO-BE	¿Se elimina comportamiento?	Razón
ResController → Hacienda	Eliminar la dependencia concreta y usar IConsultaPotreros/casos de uso.	No	El controller no debe modificar directamente el estado general.
ResController → PersistenciaService	Eliminar del controller.	No	Los guardados necesarios se trasladan a ResService/VentaService.
Alimentar desde controller	Delegar a ResService.AlimentarRes.	No	El cambio de peso y sus notificaciones se conservan.
Vender desde controller	Delegar a IVentaRes.	No	VentaService conserva la misma lógica de venta.

Solución aprobada

Cambio detallado

Estructura final del controller

ResController queda limitado a recibir y responder solicitudes MVC. Para ello utiliza ResService, las capacidades de consulta necesarias y IVentaRes para la venta. Las referencias directas a Hacienda y PersistenciaService, así como la modificación de estado y la coordinación de guardados, se eliminan del controlador y pasan a los casos de uso responsables.

El controlador conserva únicamente responsabilidades de presentación: recibir los datos HTTP, solicitar el caso de uso y devolver la misma respuesta MVC. No necesita conocer dónde vive la colección de reses ni cómo se guardan los cambios.

Alimentar: mover el guardado, no borrarlo

En el flujo de alimentación original, ResController participaba en la modificación y luego solicitaba directamente el guardado de reses. En el TO-BE, el controlador delega en ResService.AlimentarRes; el servicio conserva la modificación del peso, las notificaciones y la actualización persistente a través de IActualizacionReses. La conducta observable se mantiene, pero la presentación deja de coordinar persistencia.

Se reutiliza IActualizacionReses porque H-16 ya separó esa capacidad de actualización. Crear otra interfaz equivalente duplicaría una capacidad existente. La llamada cambia de ubicación, no de finalidad: el estado resultante continúa persistido.

Vender: usar IVentaRes y conservar las actualizaciones

En el flujo de venta, ResController delega el caso de uso a IVentaRes, cuya implementación es VentaService. El servicio conserva la responsabilidad de actualizar el estado de ventas y el estado de reses y solicita esas actualizaciones mediante IActualizacionVentas e IActualizacionReses. El controlador deja de coordinar directamente los efectos internos de persistencia.

IVentaRes ya existía y representa exactamente el caso de uso que el controller necesita, por lo que no se crea una interfaz nueva para vender. VentaService continúa siendo quien ejecuta la venta. Los guardados posteriores que antes coordinaba el controller se trasladan al servicio mediante abstracciones.

Por qué sí se crea IActualizacionVentas

Para reses ya existe IActualizacionReses, por lo que se reutiliza. Para ventas no existía un contrato estrecho equivalente: solo estaba la operación concreta GuardarVentas de PersistenciaService. Por eso se agrega IActualizacionVentas. Su responsabilidad es únicamente reflejar el estado actual de las ventas después del caso de uso; no expone todo PersistenciaService ni obliga a conocer archivos.

IActualizacionVentas expresa únicamente la capacidad de reflejar el estado actualizado de las ventas. PersistenciaService implementa ese contrato y VentaService lo consume después de ejecutar el caso de uso. De esta forma, el servicio comercial no depende de la clase concreta de persistencia ni recibe operaciones que no necesita.

Por qué no se elimina automáticamente PersistenciaService

La dependencia concreta sí desaparece del controller, pero la capacidad de guardar no desaparece del sistema. Este punto aplica la precisión establecida durante la revisión: una dependencia no se elimina solo porque queramos simplificar. Primero se determina si representa un comportamiento necesario. En H-32 los guardados forman parte del resultado interno requerido, así que se conservan y se reubican donde pertenecen.

Responsabilidades después de H-32

Elemento	Responsabilidad
ResController	Petición/respuesta MVC.
ResService	Consultas, alimentación y comportamiento correspondiente de Res según las fronteras ya refactorizadas.
VentaService / IVentaRes	Ejecutar la venta de una Res.
IConsultaPotreros	Proporcionar las consultas necesarias para localizar potrero/res.
IActualizacionReses	Conservar la actualización persistida de las reses.
IActualizacionVentas	Conservar la actualización de las ventas.
PersistenciaService	Implementar las capacidades de persistencia sin ser conocido por el controller.

Matriz explícita de inversión de dependencias

Campo exigido	H-32
Módulo de alto nivel	ResController; los casos de uso ResService y VentaService coordinan el comportamiento.
Módulo de bajo nivel / detalle concreto	Hacienda y PersistenciaService concretos.
Abstracción que desacopla	IConsultaPotreros, IVentaRes, IActualizacionReses e IActualizacionVentas.
Composition root	p_mvchacienda/Program.cs, método Main: registra

Campo exigido	H-32
	VentaService como IVentaRes, ConsultaPotreros como IConsultaPotreros y la misma PersistenciaService como IActualizacionReses/IActualizacionVentas; el contenedor construye ResController, ResService y VentaService con esas capacidades.

Qué se gana e impacto para negocio y mantenimiento

Cambiar almacenamiento o la estructura interna de Hacienda ya no obliga a modificar ResController. La alimentación y la venta quedan como casos de uso completos, incluyendo las actualizaciones necesarias. Esto reduce el radio de regresión y evita que una futura interfaz web distinta tenga que conocer la secuencia interna de guardados.

Trazabilidad final de los requisitos SOLID

Justificación de los cinco principios SOLID. El cuerpo del documento desarrolla la aplicación de SRP, OCP, LSP, ISP y DIP dentro de cada hallazgo, explicando la frontera elegida, por qué se escogió, qué se gana y qué comportamiento se preserva; por tanto, la evidencia no se limita a afirmar que un principio fue aplicado.

Localización del componente investigativo de DIP. La respuesta al requisito «módulo de alto nivel, módulo de bajo nivel, abstracción que desacopla y composition root» se encuentra dentro de cada propuesta de diseño específica que introduce o revisa una inversión de dependencia. Para localizarla con rapidez, todas esas matrices fueron resaltadas en color morado. En cada una se identifican explícitamente los cuatro campos exigidos y se precisa que el composition root efectivo del código final es p_mvcHacienda/Program.cs, método Main, mediante las registraciones de builder.Services y las configuraciones construidas allí. Cuando la corrección elimina la dependencia en lugar de invertirla (H-03), la matriz lo declara expresamente y explica por qué no corresponde conservar una abstracción artificial para esa relación.

Verificación DIP adicional del código final — SC-1 Productos Derivados

Campo exigido	SC-1 — Productos Derivados
Módulo de alto nivel	ProductoDerivadoController en la frontera MVC y VentaService en el caso de uso de venta de productos derivados.
Módulo de bajo nivel / detalle concreto	VentaService respecto al controller; InventarioProductosDerivados respecto al caso de uso; PersistenciaProductosDerivados respecto a la actualización del inventario.
Abstracción que desacopla	IVentaProductoDerivado; IInventarioProductosDerivados; IActualizacionProductosDerivados.
Composition root	p_mvcHacienda/Program.cs, método Main: registra la misma instancia de VentaService como IVentaProductoDerivado; construye una única instancia de InventarioProductosDerivados y la

	expone como <code>IVendible</code> se utiliza como abstracción de dominio para generalizar qué puede venderse (Res o <code>ProductoDerivado</code>). Su propósito principal en SC-1 es OCP/polimorfismo de dominio; no se contabiliza como una inversión técnica adicional entre un módulo de alto nivel y un detalle de infraestructura.
--	--

expone como `IVendible` se utiliza como abstracción de dominio para generalizar qué puede venderse (Res o `ProductoDerivado`). Su propósito principal en SC-1 es OCP/polimorfismo de dominio; no se contabiliza como una inversión técnica adicional entre un módulo de alto nivel y un detalle de infraestructura.

Verificación global de todas las herencias — LSP

La revisión se realizó contra el código fuente final suministrado. Se distinguieron las generalizaciones clase→clase de las realizaciones de interfaces: implementar una interfaz no se contabiliza aquí como herencia de implementación. El código contiene 8 herencias de dominio y 8 herencias técnicas de controladores hacia `Microsoft.AspNetCore.Mvc.Controller`. Las 8 herencias de dominio son las que aparecen como generalizaciones propias del modelo TO-BE.

Grupo	Superclase	Subclases	Resultado
Dominio — Res	Res	Ternero, Cebon, Novillo	3 herencias conservadas; pasan LSP
Dominio — Vacunas	Vacuna	Bacteriana, Viva	2 herencias conservadas; pasan LSP
SC-1 — Productos derivados	ProductoDerivado	Lacteo, Carne, Piel	3 herencias nuevas; pasan LSP
Framework MVC	Controller	AccountController, HomeController, PotreroController, ProductoDerivadoController, ResController, UsuarioController, VacunaController, VentaController	8 herencias técnicas; justificadas por el framework

Resultado global de las herencias vigentes del dominio: 8 de 8 pasan la verificación explícita de sustituibilidad. Además, la antigua jerarquía `Validacion` → validadores concretos no pasaba LSP y fue eliminada, como se documenta al final de esta sección.

Ternero → Res

Criterio	Verificación contra el código final
Estado	Conservada.
Por qué herencia	Ternero es una Res: participa en colecciones de Res, ventas, vacunación, alimentación y consultas mediante el contrato de la entidad base.
Por qué no composición	La identidad, Nombre, Peso, Edad, vacunas aplicadas, <code>ReglaPeso</code> y <code>TipoVendible</code> pertenecen a

	una misma entidad Res. Componer una Res dentro del subtipo duplicaría identidad y obligaría a delegar el contrato común sin aportar una responsabilidad independiente.
Precondiciones	No se fortalecen las precondiciones de los miembros heredados. Edad siempre usa el único setter Res.Edad y valida mediante la IReglaEdadRes recibida; Nombre y Peso conservan el mismo contrato de Res.
Postcondiciones	Si una asignación heredada finaliza correctamente, el estado observado por una referencia Res refleja el valor solicitado. TipoRes/TipoVendible devuelve de forma estable la categoría del subtipo.
Invariantes	La invariante de Res exige que Edad sea válida según la regla inyectada; para Ternero, ReglaEdadTernero acepta edad ≤ 12 . El subtipo no redefine Edad.
Excepciones	Las rutas comunes usan los mismos miembros de Res. Una edad inválida se rechaza desde Res.Edad con Exception y el mensaje configurado por el subtipo; no aparece un miembro Edad oculto ni NotImplementedException.
Sustituibilidad	PASA. Un Ternero utilizado como Res ejecuta los mismos miembros públicos de Res; la variación de edad/peso se compone mediante reglas y la capacidad sanitaria adicional se expresa por IReglasVacunacionRes, sin alterar el contrato base.

Cebon → Res

Criterio	Verificación contra el código final
Estado	Conservada.
Por qué herencia	Cebon mantiene una relación es-una con Res y comparte la misma identidad y comportamiento común de la entidad ganadera.
Por qué no composición	La identidad, Nombre, Peso, Edad, vacunas aplicadas, ReglaPeso y TipoVendible pertenecen a una misma entidad Res. Componer una Res dentro del subtipo duplicaría identidad y obligaría a delegar el contrato común sin aportar una responsabilidad independiente.

Precondiciones	No se fortalecen las precondiciones de los miembros heredados. Edad siempre usa el único setter Res.Edad y valida mediante la IReglaEdadRes recibida; Nombre y Peso conservan el mismo contrato de Res.
Postcondiciones	Si una asignación heredada finaliza correctamente, el estado observado por una referencia Res refleja el valor solicitado. TipoRes/TipoVendible devuelve de forma estable la categoría del subtipo.
Invariantes	La invariante de Res exige que Edad sea válida según la regla inyectada; para Cebon, ReglaEdadCebon acepta $13 \leq \text{edad} \leq 48$. El subtipo no redefine Edad.
Excepciones	Las rutas comunes usan los mismos miembros de Res. Una edad inválida se rechaza desde Res.Edad con Exception y el mensaje configurado por el subtipo; no aparece un miembro Edad oculto ni NotImplementedException.
Sustituibilidad	PASA. Un Cebon tratado como Res usa Res.Edad, Res.Peso, Res.Nombre, ReglaPeso y el contrato común sin ramas ni miembros incompatibles.

Novillo → Res

Criterio	Verificación contra el código final
Estado	Conservada.
Por qué herencia	Novillo es una Res y utiliza las mismas operaciones comunes de manejo, venta, vacunación, alimentación y consulta.
Por qué no composición	La identidad, Nombre, Peso, Edad, vacunas aplicadas, ReglaPeso y TipoVendible pertenecen a una misma entidad Res. Componer una Res dentro del subtipo duplicaría identidad y obligaría a delegar el contrato común sin aportar una responsabilidad independiente.
Precondiciones	No se fortalecen las precondiciones de los miembros heredados. Edad siempre usa el único setter Res.Edad y valida mediante la IReglaEdadRes recibida; Nombre y Peso conservan el mismo contrato de Res.

Postcondiciones	Si una asignación heredada finaliza correctamente, el estado observado por una referencia Res refleja el valor solicitado. TipoRes/TipoVendible devuelve de forma estable la categoría del subtipo.
Invariantes	La invariante de Res exige que Edad sea válida según la regla inyectada; en el código final ReglaEdadNovillo acepta edad ≥ 49 y no declara un máximo de negocio adicional. El subtipo no redefine Edad.
Excepciones	Las rutas comunes usan los mismos miembros de Res. Una edad inválida se rechaza desde Res.Edad. El constructor de Novillo conserva por compatibilidad el mensaje histórico «El ternero excedió la edad maxima»; es una anomalía textual preexistente, no una excepción adicional ni un cambio de contrato.
Sustituibilidad	PASA. Un Novillo utilizado como Res ejecuta exactamente el miembro Res.Edad y los demás miembros comunes; no existe un setter alternativo que cambie la semántica según el tipo estático de la referencia.

Bacteriana → Vacuna

Criterio	Verificación contra el código final
Estado	Conservada.
Por qué herencia	Bacteriana es una Vacuna con un dato específico de periodo de aplicación y una forma específica de acumular/validar el contador bacteriano.
Por qué no composición	Nombre, Lote, Fecha_vencimiento y Fecha_aplicacion forman la identidad común de una misma Vacuna. Usar composición obligaría a envolver una Vacuna y delegar todo el contrato base, aunque Bacteriana/Viva son categorías reales de esa entidad.
Precondiciones	Los métodos polimórficos definidos por Vacuna mantienen la misma firma. Los subtipos no exigen parámetros adicionales al invocar TipoVacuna, AcumularContador o ValidarLimite. Los requisitos particulares de construcción pertenecen a la creación del subtipo y no fortalecen una operación virtual ya prometida por Vacuna.

Postcondiciones	TipoVacuna identifica coherentemente la categoría. AcumularContador incrementa exactamente el contador correspondiente al subtipo y ValidarLimite delega la validación sanitaria apropiada mediante IReglasVacunacionRes.
Invariantes	Los miembros heredados Nombre, Lote y fechas permanecen disponibles con el mismo contrato; TipoVacuna permanece coherente con la clase concreta durante la vida de la instancia.
Excepciones	Periodo_aplicacion puede lanzar Exception durante construcción/asignación si queda fuera de los límites de ReglaVacuna; esta condición corresponde al estado específico de Bacteriana y no cambia las excepciones de los miembros comunes de Vacuna. ValidarLimite propaga las excepciones de la regla sanitaria contratada.
Sustituibilidad	PASA. Una Bacteriana puede recorrerse y operarse como Vacuna: responde TipoVacuna, AcumularContador y ValidarLimite sin comprobaciones de tipo ni operaciones no soportadas.

Viva → Vacuna

Criterio	Verificación contra el código final
Estado	Conservada.
Por qué herencia	Viva es una Vacuna con una atenuación específica y una forma específica de acumular/validar el contador de vacunas vivas.
Por qué no composición	Nombre, Lote, Fecha_vencimiento y Fecha_aplicacion forman la identidad común de una misma Vacuna. Usar composición obligaría a envolver una Vacuna y delegar todo el contrato base, aunque Bacteriana/Viva son categorías reales de esa entidad.
Precondiciones	Los métodos polimórficos definidos por Vacuna mantienen la misma firma. Los subtipos no exigen parámetros adicionales al invocar TipoVacuna, AcumularContador o ValidarLimite. Los requisitos particulares de construcción pertenecen a la creación del subtipo y no fortalecen una operación virtual ya prometida por Vacuna.

Postcondiciones	TipoVacuna identifica coherentemente la categoría. AcumularContador incrementa exactamente el contador correspondiente al subtipo y ValidarLimite delega la validación sanitaria apropiada mediante IReglasVacunacionRes.
Invariantes	Los miembros heredados Nombre, Lote y fechas permanecen disponibles con el mismo contrato; TipoVacuna permanece coherente con la clase concreta durante la vida de la instancia.
Excepciones	El constructor exige un valor enum_l_atenuaciones válido por tipo; una vez construida, los miembros comunes de Vacuna no añaden excepciones incompatibles. ValidarLimite propaga las excepciones de la regla sanitaria correspondiente.
Sustituibilidad	PASA. Una Viva usada como Vacuna satisface todos los miembros abstractos de la superclase y puede participar en colecciones y flujos que trabajan exclusivamente con Vacuna.

Lacteo → ProductoDerivado

Criterio	Verificación contra el código final
Estado	Nueva por SC-1.
Por qué herencia	Lacteo es un ProductoDerivado y reutiliza íntegramente inventario, precio, identificación y descuento; especializa únicamente su clasificación y unidad de medida.
Por qué no composición	Código, Nombre, Existencia, PrecioUnitario y Descontar forman comportamiento común real de un producto derivado. Componer un ProductoDerivado dentro de Lacteo/Carne/Piel solo añadiría un envoltorio y delegación; la diferencia legítima es TipoProducto y UnidadMedida.
Precondiciones	Los tres subtipos conservan exactamente los parámetros de construcción de ProductoDerivado y no agregan precondiciones. Código y Nombre deben ser no vacíos por contrato de la base; Descontar exige cantidad > 0 y cantidad ≤ Existencia para cualquier subtipo.
Postcondiciones	Tras construcción válida se conservan Código, Nombre, Existencia y PrecioUnitario. Tras

	Descontar(cantidad) válido, Existencia disminuye exactamente en cantidad. Los overrides devuelven de manera estable el tipo y la unidad del subtipo.
Invariantes	Código y Nombre permanecen no vacíos; Existencia no puede quedar negativa por ser uint y por la guarda de Descontar; TipoProducto/UnidadMedida permanecen coherentes con el subtipo; TipoVendible sigue delegando en TipoProducto.
Excepciones	Todos heredan las mismas ArgumentException por Código/Nombre inválidos, ArgumentOutOfRangeException para cantidad 0 e InvalidOperationException si se intenta descontar por encima de Existencia. Los overrides de clasificación/unidad no añaden excepciones.
Sustituibilidad	PASA. Un Lacteo utilizado como ProductoDerivado ejecuta sin cambios el contrato de inventario y devuelve TipoProducto="Lacteo" y UnidadMedida="Litros".

Carne → ProductoDerivado

Criterio	Verificación contra el código final
Estado	Nueva por SC-1.
Por qué herencia	Carne es un ProductoDerivado con la misma identidad, inventario y operación de descuento, diferenciándose por la clasificación y la unidad utilizada por el negocio.
Por qué no composición	Código, Nombre, Existencia, PrecioUnitario y Descontar forman comportamiento común real de un producto derivado. Componer un ProductoDerivado dentro de Lacteo/Carne/Piel solo añadiría un envoltorio y delegación; la diferencia legítima es TipoProducto y UnidadMedida.
Precondiciones	Los tres subtipos conservan exactamente los parámetros de construcción de ProductoDerivado y no agregan precondiciones. Código y Nombre deben ser no vacíos por contrato de la base; Descontar exige cantidad > 0 y cantidad ≤ Existencia para cualquier subtipo.
Postcondiciones	Tras construcción válida se conservan Código, Nombre, Existencia y PrecioUnitario. Tras

	Descontar(cantidad) válido, Existencia disminuye exactamente en cantidad. Los overrides devuelven de manera estable el tipo y la unidad del subtipo.
Invariantes	Código y Nombre permanecen no vacíos; Existencia no puede quedar negativa por ser uint y por la guarda de Descontar; TipoProducto/UnidadMedida permanecen coherentes con el subtipo; TipoVendible sigue delegando en TipoProducto.
Excepciones	Todos heredan las mismas ArgumentException por Código/Nombre inválidos, ArgumentOutOfRangeException para cantidad 0 e InvalidOperationException si se intenta descontar por encima de Existencia. Los overrides de clasificación/unidad no añaden excepciones.
Sustituibilidad	PASA. Una Carne utilizada como ProductoDerivado conserva todas las garantías de la base y devuelve TipoProducto="Carne" y UnidadMedida="Kilogramos".

Piel → ProductoDerivado

Criterio	Verificación contra el código final
Estado	Nueva por SC-1.
Por qué herencia	Piel es un ProductoDerivado y comparte el mismo ciclo de inventario y venta definido por la base; solo especializa su identidad de tipo y unidad.
Por qué no composición	Código, Nombre, Existencia, PrecioUnitario y Descontar forman comportamiento común real de un producto derivado. Componer un ProductoDerivado dentro de Lacteo/Carne/Piel solo añadiría un envoltorio y delegación; la diferencia legítima es TipoProducto y UnidadMedida.
Precondiciones	Los tres subtipos conservan exactamente los parámetros de construcción de ProductoDerivado y no agregan precondiciones. Código y Nombre deben ser no vacíos por contrato de la base; Descontar exige cantidad > 0 y cantidad ≤ Existencia para cualquier subtipo.
Postcondiciones	Tras construcción válida se conservan Código, Nombre, Existencia y PrecioUnitario. Tras Descontar(cantidad) válido, Existencia disminuye

	exactamente en cantidad. Los overrides devuelven de manera estable el tipo y la unidad del subtipo.
Invariantes	Código y Nombre permanecen no vacíos; Existencia no puede quedar negativa por ser uint y por la guarda de Descontar; TipoProducto/UnidadMedida permanecen coherentes con el subtipo; TipoVendible sigue delegando en TipoProducto.
Excepciones	Todos heredan las mismas ArgumentException por Código/Nombre inválidos, ArgumentOutOfRangeException para cantidad 0 e InvalidOperationException si se intenta descontar por encima de Existencia. Los overrides de clasificación/unidad no añaden excepciones.
Sustituibilidad	PASA. Una Piel utilizada como ProductoDerivado conserva el contrato de la base y devuelve TipoProducto="Piel" y UnidadMedida="Unidades".

Herencia técnica de framework — Controller → controladores MVC

Herencias verificadas: AccountController, HomeController, PotreroController, ProductoDerivadoController, ResController, UsuarioController, VacunaController y VentaController →

Microsoft.AspNetCore.Mvc.Controller.

Criterio	Verificación contra el código final
Por qué herencia	ASP.NET Core MVC define Controller como superclase de infraestructura para controladores con vistas. Los controladores concretos usan capacidades heredadas del framework como View(...), RedirectToAction(...), ModelState, ControllerContext y resultados MVC.
Por qué no composición	Reemplazar Controller por un objeto contenido no satisface directamente el modelo de activación y las capacidades protegidas/heredadas esperadas por MVC; obligaría a reimplementar o delegar infraestructura del framework sin beneficio de dominio.
Precondiciones	Cada acción se ejecuta bajo el mismo contexto MVC que exige Controller; los controladores no fortalecen precondiciones de métodos virtuales heredados para ser activados por el framework.
Postcondiciones	Las acciones devuelven resultados MVC compatibles (ActionResult/ActionResult o equivalentes) y mantienen válido el contexto

	heredado.
Invariantes	Cada clase permanece siendo un Controller válido para el pipeline MVC; la especialización agrega acciones y dependencias del caso de uso, no rompe el estado base del framework.
Excepciones	No se introducen excepciones incompatibles por sobrescritura de miembros base; las excepciones de negocio provienen de las acciones/casos de uso y no de una sustitución defectuosa de Controller.
Sustituibilidad	PASA como herencia técnica: el runtime MVC puede tratar cada controlador concreto como Controller. Esta generalización pertenece al framework y no se usa para modelar relaciones de negocio en el TO-BE.

Jerarquía que no pasó LSP y fue reemplazada — Validacion → validadores concretos

Criterio	Resultado y reemplazo
Estado	ELIMINADA. La jerarquía no existe en el código final.
Por qué no se conserva	Validacion/IValidarInformacion prometía validaciones para Res, Potrero, Vacuna y Venta, pero cada validador concreto solo soportaba realmente una; las operaciones restantes podían terminar en NotImplementedException.
Por qué no composición	El problema no era reutilizar estado de Validacion sino un contrato demasiado amplio. Componer esa misma clase conservaría operaciones incompatibles; tampoco existía algoritmo común que justificara una superclase.
Precondiciones	La base permitía invocar operaciones que el subtipo no podía aceptar de forma efectiva; el contrato real dependía del subtipo, por lo que no existía una precondición uniforme segura.
Postcondiciones	No podían garantizarse las postcondiciones del contrato base para operaciones no soportadas.
Invariantes	La promesa «todo validador puede validar los cuatro dominios» era falsa para las implementaciones concretas.
Excepciones	NotImplementedException constituía una

	excepción sorpresa para operaciones prometidas por la superclase/interfaz amplia, evidenciando la ruptura de sustituibilidad.
Reemplazo	IValidador<T>. ValidadorRes realiza IValidador<Res>; ValidadorPotrero realiza IValidador<Potrero>; ValidadorVacuna realiza IValidador<Vacuna>; ValidadorVenta realiza IValidador<Venta>. Cada implementación promete únicamente la operación que puede cumplir y no se conserva una jerarquía de implementación artificial.

Conclusión LSP. Las ocho herencias de dominio vigentes representan relaciones «es-un» defendibles y pasan la revisión de precondiciones, postcondiciones, invariantes y excepciones. Las tres especializaciones de Res ya no redefinen Edad; Bacteriana/Viva cumplen el contrato abstracto de Vacuna; y Lacteo/Carne/Piel no fortalecen el contrato de ProductoDerivado. La única jerarquía del diseño original identificada como no sustituible —Validacion— fue reemplazada por contratos genéricos IValidador<T>.